TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO DỰ ÁN KẾT THÚC HỌC PHẦN

HỆ THỐNG QUẢN TRỊ XƯỞNG DỊCH VỤ Ô TÔ TÍCH HỢP TRỢ LÝ TRÍ TUỆ NHÂN TẠO (GARAGETOOLS AI)

Học phần:	Ứng dụng trí tuệ nhân tạo – AIA331
Hình thức:	Báo cáo Dự án Học phần – Mã đề tài 80277
Nhóm thực hiện:	Nhóm 23
Giảng viên hướng dẫn:	TS. Nguyễn Tuấn Anh

Sinh viên thực hiện: Nguyễn Đức Dũng (Trưởng nhóm)

Đồng Quốc Khánh (Thành viên)

THÁI NGUYÊN, NĂM 2026

LỜI CẢM ƠN

Nhóm thực hiện xin bày tỏ lòng biết ơn chân thành tới Ban Giám hiệu Trường Đại học Công nghệ Thông tin và Truyền thông (ICTU), Khoa Công nghệ Thông tin cùng các thầy cô giáo trong Bộ môn đã trang bị nền tảng kiến thức chuyên môn vững chắc về kỹ nghệ phần mềm, cơ sở dữ liệu, an toàn thông tin và trí tuệ nhân tạo ứng dụng trong suốt quá trình học tập.

Đặc biệt, nhóm xin gửi lời cảm ơn sâu sắc tới **TS. Nguyễn Tuấn Anh** – Giảng viên hướng dẫn học phần *Ứng dụng trí tuệ nhân tạo (AIA331)*. Những định hướng sát sao và yêu cầu kiểm chứng thực tế của Thầy về phân quyền theo vai trò và phạm vi công việc, bảo toàn tính nhất quán của sổ cái tồn kho và tiền tệ, cơ chế duyệt kết quả AI có con người kiểm soát (*Human Review*) và nguyên tắc kiểm tra chéo dữ liệu đầu ra của mô hình ngôn ngữ lớn đã giúp nhóm hoàn thiện hệ thống **GarageTools AI** bám sát yêu cầu đề tài 80277.

Mặc dù nhóm đã nỗ lực hoàn thiện toàn bộ mã nguồn, bộ kiểm thử tự động và tài liệu báo cáo, song sản phẩm khó tránh khỏi những điểm còn có thể tối ưu hơn nữa. Nhóm rất mong nhận được sự góp ý của Quý Thầy Cô để tiếp tục hoàn thiện hệ thống trong tương lai.

Thái Nguyên, năm 2026

Nhóm sinh viên thực hiện (Nhóm 23)

Nguyễn Đức Dũng

Đồng Quốc Khánh

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Thuật ngữ tiếng Anh	Ý nghĩa tiếng Việt và ngữ cảnh trong hệ thống
ACID	Atomicity, Consistency, Isolation, Durability	Bốn tính chất bảo đảm an toàn giao dịch CSDL (thực thi qua khối `BEGIN IMMEDIATE` trên SQLite WAL).
API	Application Programming Interface	Giao diện lập trình ứng dụng (14 mô-đun JSON tại `app/routes/api/` và 7 mô-đun HTML Jinja2 tại `app/routes/`).
CSRF	Cross-Site Request Forgery	Chống giả mạo yêu cầu liên trang qua cơ chế Double-Submit Cookie (`garage_csrf` + `X-CSRF-Token`).
ERD	Entity-Relationship Diagram	Sơ đồ thực thể liên kết (21 bảng chuẩn 3NF, gồm 17 bảng nghiệp vụ lõi và 4 bảng mở rộng `shop_*`).
HMAC	Hash-based Message Auth Code	Mã xác thực thông điệp SHA-256 dùng để ký phiên đăng nhập trên cookie `garage_session`.
IDOR	Insecure Direct Object Reference	Lỗ hổng truy cập trái phép bản ghi ngang hàng; được chặn bởi hàm `duoc_gan_cong` (`technician_id == user.id`).
LLM	Large Language Model	Mô hình ngôn ngữ lớn (`openai/gpt-oss-120b` trên Groq, `gemini-2.0-flash`, OpenRouter, OpenAI `gpt-4o-mini`).
ODO	Odometer	Số công-tơ-mét đo tổng quãng đường xe đã di chuyển (`vehicles.odometer` và `odometer_at_intake`, đơn vị: km).
RBAC	Role-Based Access Control	Phân quyền theo 4 vai trò trong `security_matrix.py`: `MANAGER`, `RECEPTIONIST`, `TECHNICIAN`, `CASHIER`.
RO	Repair Order	Phiếu sửa chữa (`repair_orders`), vận hành qua máy trạng thái 6 bước từ `RECEIVED` đến `COMPLETED` hoặc `CANCELLED`.
SKU	Stock Keeping Unit	Mã định danh mặt hàng phụ tùng trong kho (`parts.sku`).
SPA / SSR	Single-Page App / Server-Side Rendering	Giao diện kép kết hợp React 19 + TanStack Start (`frontend/`) và Jinja2 SSR (`app/templates/`).
UAT	User Acceptance Testing	Kiểm thử chấp nhận người dùng (35 ca nghiệm thu theo 4 vai trò tại `USER_ACCEPTANCE_TEST.md`).
VIN	Vehicle Identification Number	Số khung định danh phương tiện (`vehicles.vin`, ràng buộc `UNIQUE`).
WAL	Write-Ahead Logging	Chế độ ghi nhật ký trước của SQLite 3 giúp đọc/ghi đồng thời ổn định và sao lưu nóng an toàn (`scripts/backup.ps1`).

MỤC LỤC

LỜI CẢM ƠN i

DANH MỤC TỪ VIẾT TẮT ii

MỞ ĐẦU vii

CHƯƠNG 1. GIỚI THIỆU BÀI TOÁN VÀ ĐỊNH HƯỚNG GIẢI PHÁP 1
1.1 Thực trạng vận hành tại xưởng sửa chữa ô tô vừa và nhỏ . . . 1
1.2 Quyết định kiểm chứng ở KT1 và Ranh giới an toàn cho AI . . . 1
1.3 Phạm vi các phân hệ và Chốt chặn kỹ thuật đã hiện thực hóa . . . 2
1.4 Kết luận chương . . . 3

CHƯƠNG 2. NỀN TẢNG CÔNG NGHỆ VÀ KHẢO SÁT GIẢI PHÁP 4
2.1 Khảo sát các phương thức quản lý xưởng dịch vụ hiện nay . . . 4
2.2 Lựa chọn ngăn xếp công nghệ (Technology Stack) . . . 4
2.3 Nguyên lý kiểm soát dữ liệu đầu vào và hậu kiểm đầu ra cho AI . . . 5
2.4 Kết luận chương . . . 6

CHƯƠNG 3. PHÂN TÍCH YÊU CẦU HỆ THỐNG VÀ KIỂM SOÁT PHÂN QUYỀN 7
3.1 Phân tích các vai trò người dùng và phạm vi nghiệp vụ . . . 7
3.2 Danh mục yêu cầu chức năng (FR) . . . 9
3.3 Danh mục yêu cầu phi chức năng (NFR) . . . 11
3.4 Kiểm soát phân quyền RBAC và Cô lập phiếu theo Kỹ thuật viên (`duoc_gan_cong`) 12
3.5 Máy trạng thái Phiếu sửa chữa (6 trạng thái) và Quy trình Human Review . . . 13
3.6 Kết luận chương . . . 14

CHƯƠNG 4. THIẾT KẾ KIẾN TRÚC, DỮ LIỆU VÀ TÍCH HỢP AI 15
4.1 Kiến trúc tổng thể 4 tầng (C4 Container) . . . 15
4.2 Thiết kế cơ sở dữ liệu quan hệ (ERD 21 bảng chuẩn 3NF) . . . 16
4.3 Thiết kế Pipeline AI 7 bước và Cơ chế Chống ảo giác giá . . . 19
4.4 Ba vòng thử nghiệm và tối ưu Prompt (`KT3_PROMPT_EXPERIMENTS.md`) . . . 19
4.5 Thiết kế kiến trúc triển khai và Hệ thống Route / API . . . 20
4.6 Kết luận chương . . . 23

CHƯƠNG 5. HIỆN THỰC HÓA HỆ THỐNG 24
5.1 Tổng quan cấu trúc mã nguồn thực tế . . . 24
5.2 Hiện thực hóa các chốt chặn nghiệp vụ và bảo mật ở Backend . . . 25
5.2.1 Ma trận Phân quyền tập trung và Cô lập phiếu theo Kỹ thuật viên . . . 26
5.2.2 Bốn chốt chặn bảo vệ Tiền tệ, Sổ cái Tồn kho và Hóa đơn . . . 26
5.3 Hiện thực hóa Giao diện Người dùng và Ảnh minh chứng thực tế . . . 27
5.4 Công thức Bất biến Số học về Tiền tệ và Sổ cái Tồn kho . . . 30
5.5 Kết luận chương . . . 30

CHƯƠNG 6. KIỂM THỬ THỰC NGHIỆM VÀ ĐỐI KHÁNG 31
6.1 Chiến lược kiểm thử đa tầng (Testing Pyramid) . . . 31
6.2 Danh mục các bộ kiểm thử tự động . . . 31
6.3 Kiểm thử đối kháng bảo mật và Toàn vẹn nghiệp vụ . . . 32
6.4 Kiểm thử biên cho Trợ lý AI và Chu trình Sao lưu / Phục hồi . . . 33

6.5 Tổng hợp kết quả nghiệm thu thực nghiệm 34
6.6 Kết luận chương 34

CHƯƠNG 7. ĐÁNH GIÁ THỰC NGHIỆM HỆ THỐNG AI VÀ HIỆU NĂNG VẬN HÀNH 35
7.1 Môi trường và Phương pháp đo kiểm thực nghiệm 35
7.2 Kết quả đối chiếu qua 3 vòng thử nghiệm cải tiến Prompt 35
7.3 Kết quả đo kiểm trên các kịch bản dữ liệu thực tế (AI01 – AI03) 36
7.4 Phân tích thời gian phản hồi (Latency Benchmark) 37
7.5 Đánh giá chi phí vận hành và Các phương án API miễn phí 37
7.6 Kết luận chương 38

CHƯƠNG 8. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 39
8.1 Tổng kết các kết quả đạt được 39
8.2 Đánh giá khách quan các giới hạn hiện tại (Chủ đích an toàn) 39
8.3 Lộ trình phát triển tiếp theo 40
8.4 Lời kết 41

TÀI LIỆU THAM KHẢO 42

PHỤ LỤC A. HƯỚNG DẪN CÀI ĐẶT, VẬN HÀNH VÀ KIỂM CHỨNG ĐỘC LẬP 43
A.1 Yêu cầu môi trường và Cấu hình biến môi trường (`.env.example`) 43
A.2 Quy trình khởi tạo CSDL, chạy ứng dụng và thực thi kiểm thử 43

PHỤ LỤC B. NHẬT KÝ SỬ DỤNG AI TRONG PHÁT TRIỂN VÀ REVIEW MÃ NGUỒN 45

PHỤ LỤC C. MA TRẬN ĐỐI CHIẾU YÊU CẦU ĐỀ TÀI 80277 VÀ TIÊU CHÍ HỌC PHẦN AIA331 47

DANH MỤC BẢNG BIỂU

1.1 Phạm vi 7 phân hệ nghiệp vụ và các chốt chặn kỹ thuật trong GarageTools AI . 3
2.1 Đối chiếu các phương thức hiện có với hệ thống GarageTools AI (Đề tài 80277) 4
3.1 Bốn vai trò người dùng, tài khoản demo và giới hạn quyền hạn trong hệ thống . 8
3.2 Danh mục yêu cầu chức năng và cơ chế kiểm chứng trong hệ thống 9
4.1 Đặc tả cấu trúc 21 bảng dữ liệu và ràng buộc toàn vẹn trong `database/schema.sql` 17
4.2 Dữ liệu đầu vào và cơ chế kiểm soát đầu ra của 3 chức năng AI 19
4.3 Danh mục các mô-đun JSON API chính trong `app/routes/api/` 22
5.1 Cấu trúc phân bổ mã nguồn thực tế của dự án `Nhom23_GarageToolsAI` 25
5.2 Ma trận phân giải quyền hạn theo 4 vai trò trong `app/services/security_matrix.py` 26
5.3 Các chốt chặn kỹ thuật bảo vệ toàn vẹn giao dịch trong `app/services/` 27
6.1 Danh mục các bộ kiểm thử tự động và kịch bản nghiệm thu trong dự án Garage AI 32
6.2 Ma trận kiểm thử đối kháng bảo mật và chốt chặn toàn vẹn dữ liệu 33
6.3 Tổng hợp kết quả thực thi kiểm thử trên toàn bộ các tầng của hệ thống 34
7.1 Tổng hợp kết quả 3 vòng thử nghiệm và cải tiến Prompt (`KT3_PROMPT_EXPERIMENTS.md`) 36
7.2 Kết quả kiểm định thực tế của 3 chức năng AI01, AI02 và AI03 36
7.3 Thống kê thời gian phản hồi thực nghiệm của các phân hệ nội bộ và cổng AI . . 37
7.4 Các chế độ cấu hình nhà cung cấp AI và chi phí vận hành tương ứng 38
8.1 Các giới hạn kỹ thuật hiện tại của hệ thống và phương án mở rộng 40
A.1 Danh sách 4 tài khoản demo phục vụ kiểm thử nghiệm thu theo 4 vai trò 44
B.1 Nhật ký sử dụng AI và các lỗi / đề xuất bất hợp lý đã được nhóm kiểm chứng, loại bỏ 46
C.1 Ma trận truy vết yêu cầu đề tài 80277 và tiêu chí đánh giá học phần AIA331 . . 47

DANH MỤC HÌNH VẼ

1.1 Sơ đồ ngữ cảnh hệ thống (System Context Diagram) của GarageTools AI . . . 2
3.1 Sơ đồ ca sử dụng tổng thể (Use Case Diagram) của hệ thống GarageTools AI . 9
3.2 Biểu đồ tuần tự (Sequence Diagram) quy trình sửa chữa khép kín và duyệt kết quả AI 12
3.3 Máy trạng thái vòng đời Phiếu sửa chữa (6 trạng thái) và Trạng thái duyệt kết quả AI 13
4.1 Sơ đồ kiến trúc thành phần chứa (C4 Container) của hệ thống GarageTools AI . 15
4.2 Sơ đồ thực thể liên kết (ERD) phân hệ Khách hàng, Phương tiện, Phiếu sửa chữa và Sổ cái Kho 16
4.3 Sơ đồ thực thể liên kết (ERD) phân hệ Người dùng, Báo giá, Hóa đơn, Bảo hành và Kết quả AI 16
4.4 Quy trình xử lý 7 bước của AI Gateway, lọc khớp giá CSDL và duyệt Human Review 19
4.5 Sơ đồ triển khai hệ thống trên máy trạm tại xưởng hoặc Docker Compose . . . 21
5.1 Ảnh minh chứng thực tế: Giao diện Quản lý và lọc Phiếu sửa chữa theo 6 trạng thái 28
5.2 Ảnh minh chứng thực tế: (Trái) Bảng điều khiển Tổng quan Dashboard; (Phải) Chi tiết Phiếu sửa chữa #1 kèm hạng mục chốt giá và Biên bản tiếp nhận xe . . 28

5.3 Ảnh minh chứng thực tế: Giao diện Kho phụ tùng và Sổ cái giao dịch đối soát thời gian thực (`Lệch sổ: 0`) 29
5.4 Ảnh minh chứng thực tế Trung tâm AI: (Trái) Kết quả mới sinh ở trạng thái *Chờ duyệt*; (Phải) Sau khi Quản lý phê duyệt chuyển sang *Đã duyệt* 29
6.1 Ảnh minh chứng kiểm thử đối kháng trên giao diện: (Trái) Hệ thống chặn thu tiền vượt quá số nợ còn phải thu; (Phải) Chặn truy cập trái thẩm quyền RBAC . 33
7.1 Biểu đồ đối chiếu thời gian phản hồi thực nghiệm trên Groq Cloud (`openai/gpt-oss-120b`) và chế độ `mock` (tất cả đều thấp hơn nhiều so với ngưỡng tối đa 10 giây). 37

MỞ ĐẦU

1. Tính cấp thiết của đề tài

Tại các xưởng sửa chữa và bảo dưỡng ô tô (garage) vừa và nhỏ hiện nay, việc quản lý phương tiện ra vào xưởng, theo dõi phụ tùng xuất kho, lập báo giá và thu tiền phần lớn vẫn dựa vào sổ tay, bảng tính Excel rời rạc hoặc trao đổi qua tin nhắn. Cách làm thủ công này thường dẫn tới bốn rủi ro lớn: (1) lệch tồn kho và xuất âm kho khi hai nhân viên cùng lấy một phụ tùng hoặc khi hủy phiếu mà quên cộng trả lại hàng; (2) sai lệch lịch sử tài chính khi sửa giá dịch vụ trong danh mục làm thay đổi cả tổng tiền của những hóa đơn cũ; (3) mất nhiều thời gian lục tìm lịch sử sửa chữa của xe qua các lần vào xưởng và giải thích kỹ thuật cho khách hàng; và (4) thiếu phân quyền rõ ràng giữa Quản lý, Lễ tân, Kỹ thuật viên và Thu ngân.

Sự phổ biến của các Mô hình Ngôn ngữ Lớn (LLM) mang lại khả năng hỗ trợ tóm tắt lịch sử xe, giải thích dịch vụ và soạn thảo báo giá nháp bằng tiếng Việt rất tự nhiên. Tuy nhiên, nếu gọi LLM một cách tự do thiếu kiểm soát, mô hình dễ rơi vào hiện tượng "ảo giác" (*hallucination*): tự bịa tên phụ tùng không có trong phiếu, tự cộng sai tổng tiền hoặc nguy hiểm hơn là tự suy đoán bệnh xe khi Kỹ thuật viên chưa tháo kiểm thực tế. Từ thực tiễn đó, nhóm lựa chọn thực hiện đề tài **"Hệ thống quản lý garage sửa chữa ô tô có tích hợp AI (GarageTools AI – Mã đề tài 80277)"** với nguyên tắc cốt lõi: **AI chỉ trình bày văn bản tư vấn ở trạng thái chờ duyệt (`PENDING`), không tự quyết định tiền và luôn có con người kiểm duyệt (*Human Review*)**.

2. Mục tiêu của đề tài

Đề tài hướng tới xây dựng một hệ thống phần mềm hoàn chỉnh đáp ứng 4 mục tiêu cụ thể:

- **Quản lý khép kín quy trình nghiệp vụ garage trên CSDL 21 bảng**: Bao phủ trọn vẹn từ Đặt lịch hẹn (`appointments`) → Tiếp nhận xe (`intake_records`) → Phiếu sửa chữa 6 trạng thái (`repair_orders`) → Báo giá chính thức (`quotations`) → Sổ cái tồn kho (`stock_transactions`) → Nghiệm thu → Hóa đơn (`invoices`) & Thanh toán (`payments`) → Bảo hành (`warranties`).
- **Bảo vệ tuyệt đối tiền tệ, tồn kho và phân quyền**: Lưu toàn bộ cột tiền bằng số nguyên `INTEGER` (đồng VNĐ), chốt cứng đơn giá `unit_price` lúc thêm hạng mục, bọc giao dịch trong khối `BEGIN IMMEDIATE` giữ bất biến `parts.stock` $=$ $\sum$ `stock_transactions.qty`, kết hợp phân quyền 4 vai trò (`security_matrix.py`) và cô lập Kỹ thuật viên theo phiếu được phân công (`duoc_gan_cong`).
- **Tích hợp 3 chức năng AI bám sát CSDL kèm Human Review**: Xây dựng `app/ai_gateway.py` hỗ trợ 7 cấu hình nhà cung cấp (`mock`, `groq`, `openrouter`, `gemini`, `openai`, `9router`, `custom`) cho 3 tác vụ: Tóm tắt lịch sử sửa chữa (AI01 – `history`), Giải thích dịch vụ (AI02 – `explain`) và Báo giá nháp JSON có lọc khớp giá CSDL (AI03 – `quote`).
- **Kiểm chứng tự động 100%**: Đạt tuyệt đối **231/231** ca kiểm thử tự động Back-end (`tests/run_all.py`), **28/28** ca kiểm thử Frontend (`vitest`), **31/31** ca Smoke (`scripts/smoke.ps1`), **32/32** ca E2E (`scripts/e2e.ps1`) và **35/35** ca UAT

(`USER_ACCEPTANCE_TEST.md`).

3. Đối tượng và phạm vi thực hiện

- **Đối tượng nghiên cứu**: Quy trình vận hành xưởng sửa chữa ô tô độc lập; kỹ thuật đảm bảo toàn vẹn giao dịch trên SQLite 3 (chế độ WAL + `BEGIN IMMEDIATE`); kỹ thuật thiết kế Prompt tách rời mã nguồn (`config/prompts/*.txt`) và lọc hậu kiểm kết quả JSON từ LLM.
- **Phạm vi sản phẩm**: Backend Python FastAPI (`GarageAI_Project/app/`), Frontend React 19 + TanStack Start 16 màn hình (`frontend/`) kèm giao diện Jinja2, CSDL 21 bảng và bộ kịch bản Docker Compose, kiểm thử, sao lưu/khôi phục.

4. Bố cục báo cáo

Ngoài phần Mở đầu, Danh mục từ viết tắt và 3 Phụ lục, nội dung báo cáo gồm 8 chương: **Chương 1** giới thiệu bài toán và định hướng giải pháp; **Chương 2** khảo sát giải pháp và lựa chọn công nghệ; **Chương 3** phân tích yêu cầu hệ thống, phân quyền RBAC và máy trạng thái; **Chương 4** thiết kế kiến trúc 4 tầng, CSDL 21 bảng và quy trình xử lý AI; **Chương 5** trình bày chi tiết hiện thực hóa hệ thống; **Chương 6** trình bày kết quả kiểm thử tự động và kiểm thử đối kháng; **Chương 7** đánh giá thực nghiệm 3 vòng cải tiến Prompt, độ trễ mô hình thật và chi phí vận hành; **Chương 8** tổng kết kết quả, thẳng thắn chỉ ra các giới hạn hiện tại và hướng phát triển.

CHƯƠNG 1

GIỚI THIỆU BÀI TOÁN VÀ ĐỊNH HƯỚNG GIẢI PHÁP

1.1 Thực trạng vận hành tại xưởng sửa chữa ô tô vừa và nhỏ

Tại một garage sửa chữa ô tô, luồng công việc hàng ngày diễn ra xoay quanh sự phối hợp của 4 vị trí: **Lễ tân dịch vụ** (`RECEPTIONIST`) đón khách, ghi nhận tình trạng xe lúc vào xưởng và lập báo giá; **Kỹ thuật viên** (`TECHNICIAN`) trực tiếp kiểm tra, lấy phụ tùng sửa chữa và ghi nhật ký kỹ thuật; **Quản lý xưởng** (`MANAGER`) duyệt báo giá, giám sát danh mục, lập hóa đơn và kiểm duyệt các nội dung tư vấn; và **Thu ngân** (`CASHIER`) thu tiền, đối soát công nợ.

Qua phân tích quy trình thực tế ở giai đoạn KT1, nhóm xác định 3 rủi ro lớn nhất thường xảy ra khi xưởng thiếu một hệ thống phần mềm kiểm soát chặt chẽ:

1. **Lệch tồn kho và xuất âm kho khi thao tác đồng thời**: Nếu phần mềm chỉ đọc số lượng tồn kho ra biến Python rồi mới chạy lệnh `UPDATE`, khi hai nhân viên cùng bấm xuất món phụ tùng cuối cùng sẽ dẫn tới tồn kho bị âm. Tương tự, nếu sửa trực tiếp cột `parts.stock` mà không qua sổ cái giao dịch (`stock_transactions`), hoặc khi hủy phiếu sửa chữa mà quên cộng trả lại hàng, số liệu kho sẽ nhanh chóng lệch khỏi thực tế.
2. **Sai lệch lịch sử tài chính khi thay đổi bảng giá**: Nếu đơn giá hạng mục trong phiếu sửa chữa chỉ liên kết động (*join* trực tiếp) về bảng danh mục dịch vụ `services` hoặc phụ tùng `parts`, thì mỗi khi Quản lý cập nhật giá niêm yết mới, tổng tiền của cả những phiếu sửa chữa và hóa đơn đã thanh toán trong quá khứ sẽ bị thay đổi theo.
3. **Rủi ro "ảo giác" khi dùng AI thiếu rào chắn**: Nếu nhân viên dùng các chatbot AI tự do để tóm tắt lịch sử xe hoặc soạn báo giá, mô hình ngôn ngữ có thể tự suy diễn nguyên nhân hỏng hóc khi chưa kiểm tra xe, tự thêm hạng mục không có trong phiếu hoặc cộng nhầm tổng tiền báo giá.

1.2 Quyết định kiểm chứng ở KT1 và Ranh giới an toàn cho AI

Ngay tại bước phân tích KT1 (được ghi lại trong `KT1_KT2_AI_USAGE.md`), khi đặt câu hỏi cho công cụ AI về các tính năng nên có trong phần mềm quản lý garage, AI đã gợi ý 4 tính năng gồm cả *"Tự động chẩn đoán bệnh xe từ mô tả triệu chứng hoặc tiếng động cơ"*. Sau khi thảo luận và đối chiếu với thực tế kỹ thuật, nhóm đã **chủ động loại bỏ hoàn toàn tính năng AI tự chẩn đoán lỗi xe**, bởi việc kết luận hỏng hóc trên ô tô bắt buộc phải do Kỹ thuật viên kiểm tra thực tế tại khoang máy; để AI phán đoán mò sẽ gây rủi ro mất an toàn và nằm ngoài dữ liệu kiểm chứng được của CSDL.

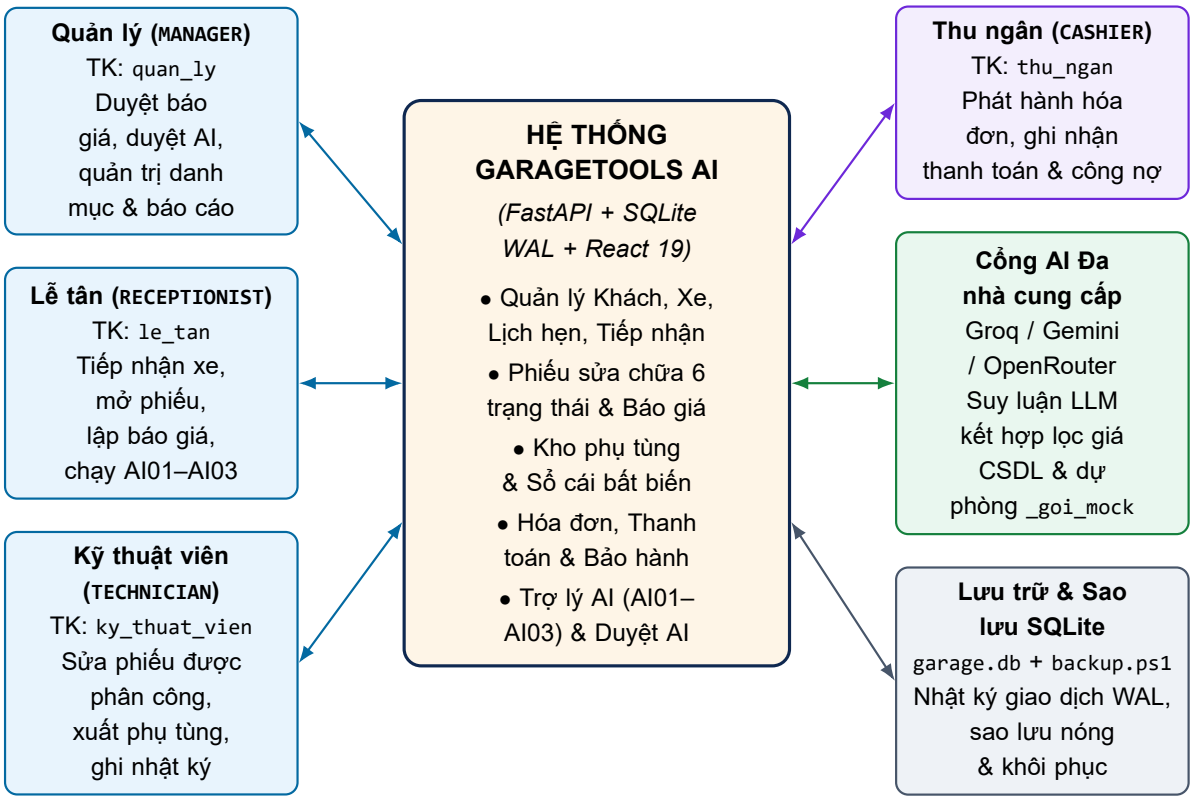
Từ đó, hệ thống **GarageTools AI** giới hạn đúng **3 chức năng AI an toàn, bám sát 100% dữ liệu có sẵn trong CSDL**:

- **AI01 – Tóm tắt lịch sử sửa chữa (`history`)**: Tổng hợp các lần sửa chữa trước đó của một phương tiện thành đúng 4 mục ngắn gọn cho Lễ tân nắm bắt nhanh khi xe quay lại

xưởng; nếu xe mới chưa từng sửa chữa (`vehicle_id=6`), AI báo rõ chưa có lịch sử và tuyệt đối không suy đoán.

- **AI02 – Giải thích dịch vụ cho khách hàng (`explain`)**: Diễn giải tên dịch vụ, đơn giá niêm yết (`services.price`) và mô tả kỹ thuật thành 3 mục bằng ngôn ngữ đời thường dễ hiểu cho chủ xe; xử lý an toàn cả ca biên dịch vụ trống mô tả (`description IS NULL`, như `service_id=7`).
- **AI03 – Báo giá nháp JSON (`quote`)**: Sắp xếp lại các hạng mục đã ghi nhận trong phiếu sửa chữa (hoặc báo giá đang mở) kèm lời tóm tắt và lưu ý; tầng Backend (`app/ai_gateway.py`) lọc bỏ mọi dòng AI tự bịa hoặc lệch giá và luôn cưỡng bức tổng tiền bằng `backend_total`.

Hình 1.1 thể hiện sơ đồ ngữ cảnh tổng thể của hệ thống **GarageTools AI**.



Hình 1.1: Sơ đồ ngữ cảnh hệ thống (System Context Diagram) của GarageTools AI

1.3 Phạm vi các phân hệ và Chốt chặn kỹ thuật đã hiện thực hóa

Bảng 1.1 tổng hợp 7 phân hệ nghiệp vụ đã được hoàn thiện trong dự án cùng các cơ chế bảo vệ toàn vẹn tương ứng.

Bảng 1.1: Phạm vi 7 phân hệ nghiệp vụ và các chốt chặn kỹ thuật trong GarageTools AI

Phân hệ hệ thống	**Phạm vi chức năng đã hiện thực hóa**	**Chốt chặn an toàn & Toàn vẹn dữ liệu**
1. Xác thực & Phân quyền (`auth`, `security_matrix`)	Đăng nhập mật khẩu băm Argon2id, cookie phiên `garage_session` ký HMAC, phân quyền 4 vai trò trên 16 màn hình.	Tăng `auth_version` để hủy mọi phiên cũ khi đổi mật khẩu/khóa user; CSRF 2 lớp; cô lập KTV qua `duoc_gan_cong`.
2. Khách hàng, Xe & Lịch hẹn (`parties.py`)	Quản lý khách hàng (`customers`), phương tiện (`vehicles`), lịch hẹn (`appointments`) 4 trạng thái.	Ràng buộc `phone UNIQUE`, `plate UNIQUE`, `vin UNIQUE`; kiểm tra chủ xe khớp với phương tiện khi đặt lịch.
3. Tiếp nhận & Phiếu sửa chữa (`orders.py`)	Biên bản tiếp nhận (`intake_records` 1–1), vòng đời Phiếu sửa chữa 6 trạng thái, nhật ký kỹ thuật và nghiệm thu.	Khóa `BEGIN IMMEDIATE` chặn mở 2 phiếu đang chạy cho cùng 1 xe; chốt cứng `unit_price` lúc thêm hạng mục.
4. Báo giá chính thức (`quotations.py`)	Lập báo giá (`quotations`, `quotation_items`) qua 5 trạng thái; duyệt và áp dụng vào phiếu sửa chữa.	Áp dụng *idempotent* đúng 1 lần qua cặp cột `applied_at`/`applied_by`, tự động trừ kho qua sổ cái.
5. Danh mục & Sổ cái Tồn kho (`catalog.py`, `stock.py`)	Quản lý dịch vụ (`services`), phụ tùng (`parts`) và sổ cái tồn kho bất biến (`stock_transactions`).	Ràng buộc `CHECK XOR` dịch vụ $\oplus$ phụ tùng; bất biến `stock` $= \sum$ `qty`; tự hoàn kho (`RETURN`) khi hủy phiếu.
6. Hóa đơn, Thanh toán & Bảo hành (`billing.py`, `warranties.py`)	Lập hóa đơn (`invoices` 1–1), thu tiền đa phương thức (`payments`), hủy khoản thu nhầm, cấp bảo hành (`warranties`).	Ràng buộc `CHECK (paid <= total)`; chặn thu vượt nợ trong khối `BEGIN IMMEDIATE` (HTTP 409); bảo hành theo hạng mục.
7. Trợ lý AI & Human Review (`ai_gateway.py`, `ai.py`)	Cung cấp AI01 (`history`), AI02 (`explain`), AI03 (`quote`) hỗ trợ 7 cấu hình provider kèm `_goi_mock`.	Lưu trạng thái `PENDING` trong `ai_results`; chỉ `MANAGER` (`ai.review`) được duyệt và chặn duyệt lại lần 2.

1.4 Kết luận chương

Chương 1 đã trình bày bối cảnh thực tế tại xưởng sửa chữa ô tô, lý giải quyết định loại bỏ tính năng AI tự chẩn đoán lỗi xe để giữ lại 3 chức năng AI an toàn, và khái quát 7 phân hệ cốt lõi của hệ thống. Chương 2 tiếp theo sẽ khảo sát các giải pháp hiện có và trình bày nền tảng công nghệ của dự án.

CHƯƠNG 2

NỀN TẢNG CÔNG NGHỆ VÀ KHẢO SÁT GIẢI PHÁP

2.1 Khảo sát các phương thức quản lý xưởng dịch vụ hiện nay

Để xác định rõ phạm vi và giá trị thực tế của hệ thống **GarageTools AI**, nhóm đã khảo sát và đối chiếu ba cách quản lý đang được dùng phổ biến tại các garage sửa chữa ô tô vừa và nhỏ: (1) Ghi chép sổ sách kết hợp bảng tính Excel và nhắn tin Zalo; (2) Phần mềm ERP/DMS thương mại quy mô lớn; và (3) Sử dụng trực tiếp các công cụ hội thoại AI tự do trên trình duyệt (ChatGPT, Claude, Gemini Web). Kết quả đối chiếu được tổng hợp tại Bảng 2.1.

Bảng 2.1: Đối chiếu các phương thức hiện có với hệ thống GarageTools AI (Đề tài 80277)

Tiêu chí đánh giá	**Sổ sách / Excel & Nhắn tin**	**Phần mềm ERP/DMS truyền thống**	**Chatbot AI tự do (Web Chat)**	**Hệ thống GarageTools AI (Đề tài 80277)**
Quản lý vòng đời Phiếu sửa chữa	Rời rạc, dễ thất lạc phiếu và sai lệch trạng thái.	Đầy đủ nhưng quy trình phức tạp, chi phí triển khai cao.	Không quản lý được trạng thái nghiệp vụ.	Máy trạng thái 6 bước chặt chẽ + Biên bản tiếp nhận 1–1 + Nghiệm thu có kiểm tra.
Kiểm soát tồn kho & Sổ cái kho	Cập nhật tay, dễ lệch tồn kho thực tế.	Có quản lý kho nhưng cấu hình nặng nề.	Không kết nối kho thực tế; dễ bịa tên phụ tùng.	Giao dịch `BEGIN IMMEDIATE` + Sổ cái tồn kho bất biến `stock_transactions`.
Phân quyền nhân sự & Cô lập KTV	Không có phân quyền; ai mở file cũng sửa được.	Có phân quyền theo phòng ban.	Không có phân quyền nội bộ xưởng.	RBAC 4 vai trò + Cô lập phiếu KTV (`duoc_gan_cong`) + Hủy phiên tức thì.
Trợ lý AI & Chống bịa đơn giá	Không hỗ trợ tự động hóa ngôn ngữ.	Đa số chưa tích hợp LLM tiếng Việt.	Dễ bịa đơn giá, cộng sai tiền, không nắm lịch sử xe.	Tích hợp AI01–AI03 đọc trực tiếp CSDL + Lọc khớp giá + ép `backend_total`.
Kiểm duyệt kết quả AI (Human Review)	Không áp dụng.	Không áp dụng cho AI.	Người dùng tự sao chép, không lưu vết kiểm tra.	Lưu bảng `ai_results` (`PENDING`), bắt buộc Quản lý duyệt (`ai.review`).

2.2 Lựa chọn ngăn xếp công nghệ (Technology Stack)

Hệ thống được thiết kế theo hướng gọn nhẹ, dễ cài đặt và vận hành tại garage nhưng đảm bảo đầy đủ các chốt chặn toàn vẹn dữ liệu:

1. **Tầng Backend – Python 3.10+ & FastAPI (`app/`)**: FastAPI được sử dụng cùng

Pydantic v2 (`app/models.py`) để kiểm tra chặt dữ liệu đầu vào/ra và tách lớp rõ ràng: tầng định tuyến (`app/routes/` và `app/routes/api/`) → tầng dịch vụ nghiệp vụ (`app/services/`) → tầng truy vấn tham số hóa (`app/crud.py`) → tầng kết nối cơ sở dữ liệu (`app/core/db.py`). Ứng dụng phục vụ đồng thời giao diện Jinja2 tích hợp sẵn và hệ thống JSON API `/api/*` cho giao diện React.

2. **Tầng Cơ sở dữ liệu – SQLite 3 bật chế độ WAL (`garage.db`)**: Với quy mô một xưởng dịch vụ độc lập, SQLite 3 khi bật `PRAGMA journal_mode = WAL`, `PRAGMA foreign_keys = ON` và `busy_timeout` cho phép đọc/ghi ổn định và sao lưu trực tuyến dễ dàng. Mọi thao tác ảnh hưởng đến tiền hoặc tồn kho đều được gói trong một giao dịch `BEGIN IMMEDIATE` và kiểm tra điều kiện ngay trong câu lệnh `UPDATE` để loại bỏ lỗi chạy đua (*race condition*).
3. **Tầng Giao diện – React 19 + TanStack Start + Tailwind CSS v4 (`frontend/`)**: Giao diện chính gồm 16 màn hình nghiệp vụ xây dựng bằng React 19, TanStack Start, TypeScript và bộ thành phần `shadcn/ui`, kết nối trực tiếp tới `/api/*` qua khách HTTP duy nhất (`frontend/src/services/api.ts`) có tự động đính kèm CSRF token.
4. **Tầng Bảo mật Phiên – Argon2id, Cookie ký HMAC, `auth_version` & CSRF 2 lớp**: Mật khẩu người dùng được băm bằng **Argon2id** (tự động chuyển sang PBKDF2-SHA256 200.000 vòng lặp nếu môi trường thiếu thư viện C). Phiên đăng nhập được lưu trong cookie `garage_session` (`HttpOnly`) ký HMAC-SHA256 kèm trường `auth_version`. Khi đổi mật khẩu hoặc khóa tài khoản, cột `users.auth_version` tăng lên 1 làm mọi phiên cũ mất hiệu lực ngay lập tức. Mọi yêu cầu sửa đổi dữ liệu đều phải gửi kèm `X-CSRF-Token` khớp với cookie `garage_csrf`.
5. **Tầng AI Gateway Đa nhà cung cấp (`app/ai_gateway.py`)**: Giao tiếp qua chuẩn OpenAI-compatible (`/chat/completions`), hỗ trợ 7 cấu hình: `mock` (mặc định khi chạy kiểm thử), `groq` (`openai/gpt-oss-120b`), `openrouter`, `gemini` (`gemini-2.0-flash`), `openai` (`gpt-4o-mini`), `9router` và `custom`.

2.3 Nguyên lý kiểm soát dữ liệu đầu vào và hậu kiểm đầu ra cho AI

Khi tích hợp mô hình ngôn ngữ lớn vào phần mềm quản lý xưởng có liên quan tới báo giá và tiền bạc, nếu chỉ dặn mô hình bằng câu chữ trong prompt là "không được bịa giá" thì vẫn có rủi ro mô hình cộng nhầm hoặc tự thêm hạng mục. Vì vậy, nhóm thiết kế cơ chế kiểm soát hai đầu trong `app/ai_gateway.py`:

- **Đầu vào lấy hoàn toàn từ CSDL**: Backend truy vấn trực tiếp lịch sử xe, danh mục dịch vụ (kèm đơn giá niêm yết `services.price`) hoặc danh sách hạng mục của phiếu sửa chữa (`repair_order_items`, hoặc lấy từ báo giá `quotations` đang mở nếu phiếu chưa đồng bộ), cắt giới hạn độ dài an toàn (`AI_MAX_CONTEXT = 6000`) rồi ghép vào các tệp mẫu tại `config/prompts/*.txt`.
- **Đầu ra được lọc và đối chiếu lại ở Backend**: Với chức năng AI03 (`quote`), sau khi tách JSON từ phản hồi của mô hình (`_phan_tich_json`), hàm `bao_gia_nhap` kiểm tra từng dòng trong mảng `items` với tập hợp dòng gốc $\mathcal{V} = \{(\text{name}_i, \text{qty}_i, \text{unit_price}_i)\}$ đọc từ

CSDL:

$$\text{HopLe}(x) \iff (\text{strip}(x.\text{name}),\ \text{int}(x.\text{qty}),\ \text{int}(x.\text{unit_price})) \in \mathcal{V} \tag{2.1}$$

Bất kỳ dòng nào AI tự thêm hoặc tự sửa số lượng/đơn giá đều bị loại bỏ; nếu sau khi lọc mà danh sách trống, hệ thống giữ nguyên danh sách gốc của CSDL và luôn ép trường `total` bằng đúng `backend_total` do Backend tự cộng.

2.4 Kết luận chương

Chương 2 đã đối chiếu giải pháp đề xuất với các công cụ hiện có, trình bày lý do lựa chọn ngăn xếp công nghệ FastAPI – SQLite WAL – React 19 và giải thích cơ chế hậu kiểm chống sai lệch số liệu khi gọi LLM. Chương 3 tiếp theo sẽ đặc tả chi tiết các yêu cầu chức năng, phi chức năng, ma trận phân quyền và máy trạng thái của hệ thống.

CHƯƠNG 3

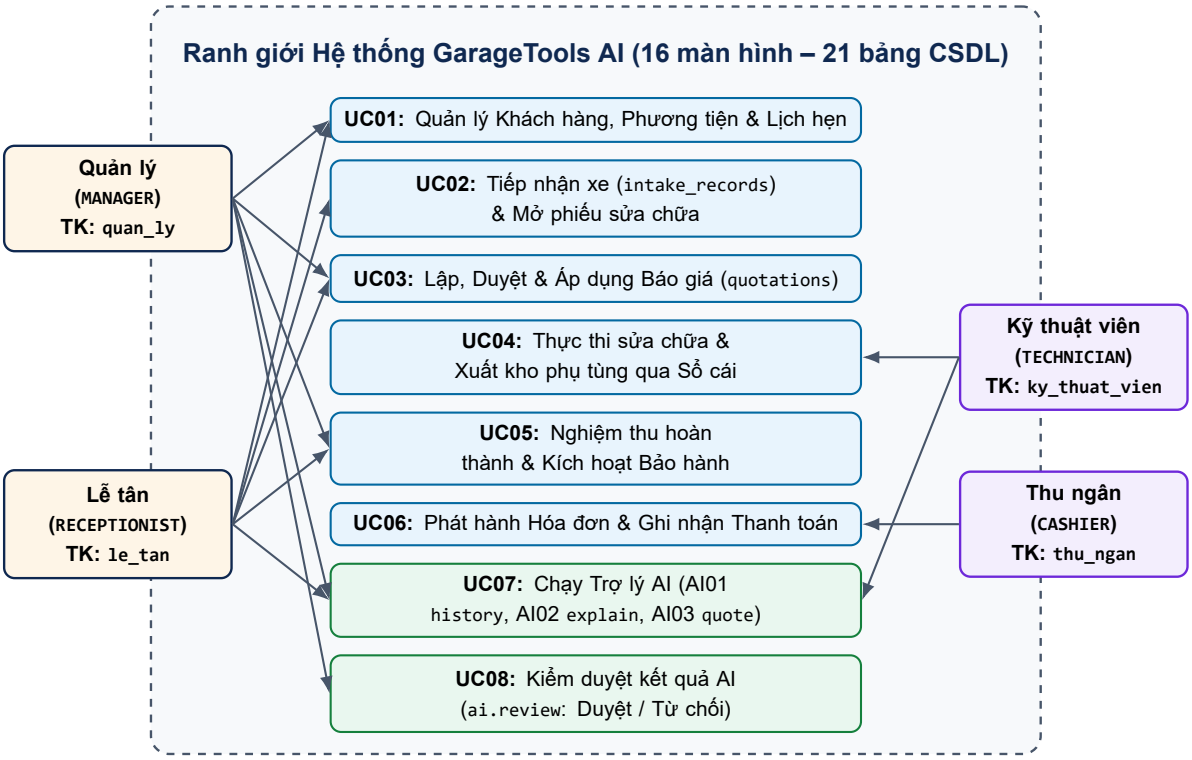
PHÂN TÍCH YÊU CẦU HỆ THỐNG VÀ KIỂM SOÁT PHÂN QUYỀN

3.1 Phân tích các vai trò người dùng và phạm vi nghiệp vụ

Trong giai đoạn phân tích KT1, khi tham vấn công cụ AI về các tác nhân của hệ thống, AI từng gợi ý 5 vai trò bao gồm cả Khách hàng đăng nhập. Tuy nhiên, đối chiếu với thực tế vận hành tại xưởng và phạm vi đề tài 80277, nhóm quyết định chỉ giữ **4 vai trò nhân sự đăng nhập nội bộ** trong bảng `users` (`MANAGER`, `RECEPTIONIST`, `TECHNICIAN`, `CASHIER`), còn Khách hàng được quản lý như một thực thể dữ liệu tại bảng `customers`. Chi tiết mục tiêu và giới hạn quyền của từng vai trò được trình bày tại Bảng 3.1 và sơ đồ Use Case tại Hình 3.1.

Bảng 3.1: Bốn vai trò người dùng, tài khoản demo và giới hạn quyền hạn trong hệ thống

Vai trò & Tài khoản	**Nhiệm vụ nghiệp vụ chính**	**Giới hạn quyền hạn (`security_matrix.py`)**
Quản lý xưởng (`MANAGER` TK: `quan_ly`)	Quản trị người dùng (`user.manage`), sửa danh mục dịch vụ/phụ tùng (`catalog.edit`), duyệt báo giá (`quote.approve`), lập hóa đơn (`invoice.manage`), duyệt kết quả AI (`ai.review`) và xem báo cáo.	Toàn quyền điều hành; là vai trò duy nhất có quyền phê duyệt hoặc từ chối kết quả AI (`APPROVED`/`REJECTED`) và duyệt báo giá chính thức.
Lễ tân dịch vụ (`RECEPTIONIST` TK: `le_tan`)	Quản lý khách hàng, phương tiện, lịch hẹn; tiếp nhận xe 1-chạm và lập biên bản (`intake_records`); điều phối trạng thái phiếu (`order.progress`), phân công KTV; lập báo giá nháp; lập hóa đơn khi phiếu hoàn thành (`invoice.manage`); nhập/điều chỉnh kho; tạo bảo hành; chạy AI01, AI02, AI03.	Được gọi cả 3 chức năng AI (`ai.history`, `ai.explain`, `ai.quote`) nhưng **bị chặn** quyền duyệt kết quả AI (`ai.review`) và duyệt báo giá (`quote.approve`).
Kỹ thuật viên (`TECHNICIAN` TK: `ky_thuat_vien`)	Xem phiếu sửa chữa được giao, cập nhật trạng thái kiểm tra/sửa chữa và ghi chú kỹ thuật (`order.progress`), thêm/gỡ hạng mục dịch vụ và phụ tùng (`order.item`), ghi nhật ký kỹ thuật (`repair_logs`) và gọi AI03 (`ai.quote`).	**Cô lập dữ liệu hoàn toàn theo phiếu được giao (`duoc_gan_cong`)**: Chỉ nhìn thấy và thao tác trên phiếu có `technician_id == user.id` tại cả Tổng quan, Lịch sử và Phiếu sửa chữa.
Thu ngân (`CASHIER` TK: `thu_ngan`)	Tra cứu phiếu sửa chữa và hóa đơn, ghi nhận thanh toán từng phần hoặc toàn bộ (`payment.record`) và hủy khoản thu nhầm.	Chỉ có quyền ghi ở phân hệ Thanh toán; bị chặn (HTTP 403) khi cố tạo/sửa phiếu sửa chữa, chỉnh kho hoặc chạy AI.



Hình 3.1: Sơ đồ ca sử dụng tổng thể (Use Case Diagram) của hệ thống GarageTools AI

3.2 Danh mục yêu cầu chức năng (FR)

Hệ thống hiện thực hóa đầy đủ 14 nhóm yêu cầu chức năng (FR01–FR14) bao phủ toàn bộ quy trình vận hành garage và 3 chức năng trợ lý AI, được tổng hợp cùng kết quả kiểm chứng trong Bảng 3.2.

Bảng 3.2: Danh mục yêu cầu chức năng và cơ chế kiểm chứng trong hệ thống

Mã	Phân hệ	Mô tả yêu cầu nghiệp vụ	Cơ chế hiện thực & Kiểm chứng	Trạng thái
FR01	Xác thực	Đăng nhập bằng tài khoản/mật khẩu; băm Argon2id; ký phiên HMAC và hủy phiên tức thì khi đổi mật khẩu/khóa user.	Kiểm tra `is_active = 1` và đối chiếu `auth_version`; bảo vệ mọi form/API bằng CSRF token 2 lớp.	ĐÃ KIỂM THỬ
FR02	Phân quyền	Kiểm soát truy cập theo 4 vai trò (RBAC) từ một nguồn duy nhất và cô lập phiếu theo Kỹ thuật viên.	Định nghĩa tập trung tại `app/services/security_matrix.py`; chặn vượt quyền dọc/ngang với HTTP 403.	ĐÃ KIỂM THỬ

Mã	Phân hệ	Mô tả yêu cầu nghiệp vụ	Cơ chế hiện thực & Kiểm chứng	Trạng thái
FR03	Khách hàng	Quản lý hồ sơ khách hàng (`customers`): thêm, sửa, xóa có kiểm tra khóa ngoại, tìm kiếm & phân trang.	Ràng buộc `phone UNIQUE`; `ON DELETE RESTRICT` chặn xóa khách đã có xe hoặc lịch hẹn.	ĐÃ KIỂM THỬ
FR04	Phương tiện	Quản lý hồ sơ xe (`vehicles`) theo biển số, hãng, dòng, năm sản xuất, số khung VIN và số công-tơ-mét (`odometer`).	Ràng buộc `plate UNIQUE`, `vin UNIQUE`, `year BETWEEN 1900 AND 2100`, `odometer >= 0`.	ĐÃ KIỂM THỬ
FR05	Lịch hẹn	Đặt lịch hẹn dịch vụ (`appointments`) và quản lý 4 trạng thái (`SCHEDULED`, `CONFIRMED`, `CANCELLED`, `DONE`).	Kiểm tra `customer_id` khớp với chủ sở hữu của `vehicle_id`; chặn đặt trùng giờ cho cùng một xe.	ĐÃ KIỂM THỬ
FR06	Tiếp nhận xe	Ghi biên bản tiếp nhận xe (`intake_records`) gắn 1–1 với phiếu sửa chữa.	Lưu `odometer_at_intake`, `fuel_level`, tình trạng ngoại thất (`exterior_condition`) và phụ kiện kèm theo.	ĐÃ KIỂM THỬ
FR07	Phiếu sửa chữa	Quản lý vòng đời Phiếu sửa chữa (`repair_orders`) qua 6 trạng thái, nhật ký kỹ thuật và nghiệm thu.	Chặn mở 2 phiếu đang chạy cho cùng 1 xe trong khối `BEGIN IMMEDIATE`; chặn hoàn thành khi phiếu chưa có hạng mục.	ĐÃ KIỂM THỬ
FR08	Báo giá	Lập báo giá chính thức (`quotations`, `quotation_items`) qua 5 trạng thái và áp dụng vào phiếu sửa chữa.	Áp dụng *idempotent* đúng 1 lần qua cột `applied_at`/`applied_by`; tự động trừ kho qua sổ cái khi áp dụng.	ĐÃ KIỂM THỬ
FR09	Kho & Sổ cái	Quản lý danh mục dịch vụ (`services`), phụ tùng (`parts`) và sổ cái tồn kho (`stock_transactions`).	Bất biến `parts.stock` $= \sum$ `qty`; tự động hoàn kho (`RETURN`) khi gỡ hạng mục hoặc hủy phiếu.	ĐÃ KIỂM THỬ
FR10	Hóa đơn & Thu	Lập hóa đơn (`invoices`) cho phiếu đã hoàn thành, thu tiền nhiều đợt (`payments`) và hủy khoản thu nhầm.	Ràng buộc `CHECK (paid <= total)`; kiểm tra chống thu vượt nợ trong khối `BEGIN IMMEDIATE` (HTTP 409).	ĐÃ KIỂM THỬ

Mã	Phân hệ	Mô tả yêu cầu nghiệp vụ	Cơ chế hiện thực & Kiểm chứng	Trạng thái
FR11	Bảo hành	Cấp và theo dõi bảo hành theo từng hạng mục dịch vụ hoặc phụ tùng (`warranties`).	Ràng buộc `CHECK XOR` giữa `SERVICE` và `PART`; chỉ tạo được cho phiếu `COMPLETED`.	ĐÃ KIỂM THỬ
FR12	AI01 & AI02	AI01 (`history`) tóm tắt lịch sử xe 4 mục; AI02 (`explain`) giải thích dịch vụ 3 mục cho khách hàng.	Xử lý an toàn cả ca biên xe mới chưa có lịch sử (`vehicle_id=6`) và dịch vụ trống mô tả (`service_id=7`).	ĐÃ ĐO KIỂM
FR13	AI03 Báo giá	AI03 (`quote`) sinh báo giá nháp JSON từ hạng mục của phiếu sửa chữa hoặc báo giá đang mở.	Lọc chỉ giữ dòng khớp Tên, Số lượng, Đơn giá với CSDL; ép `total == backend_total`; tự động fallback `_goi_mock`.	ĐÃ ĐO KIỂM
FR14	Human Review	Lưu toàn bộ kết quả AI vào bảng `ai_results` ở trạng thái chờ duyệt và kiểm duyệt bởi Quản lý.	Khởi tạo `review_status='PENDING'`; chỉ `MANAGER` (`ai.review`) được duyệt và chặn duyệt lại lần 2.	ĐÃ KIỂM THỬ

3.3 Danh mục yêu cầu phi chức năng (NFR)

Hệ thống đáp ứng 6 yêu cầu phi chức năng cốt lõi (NFR01–NFR06):

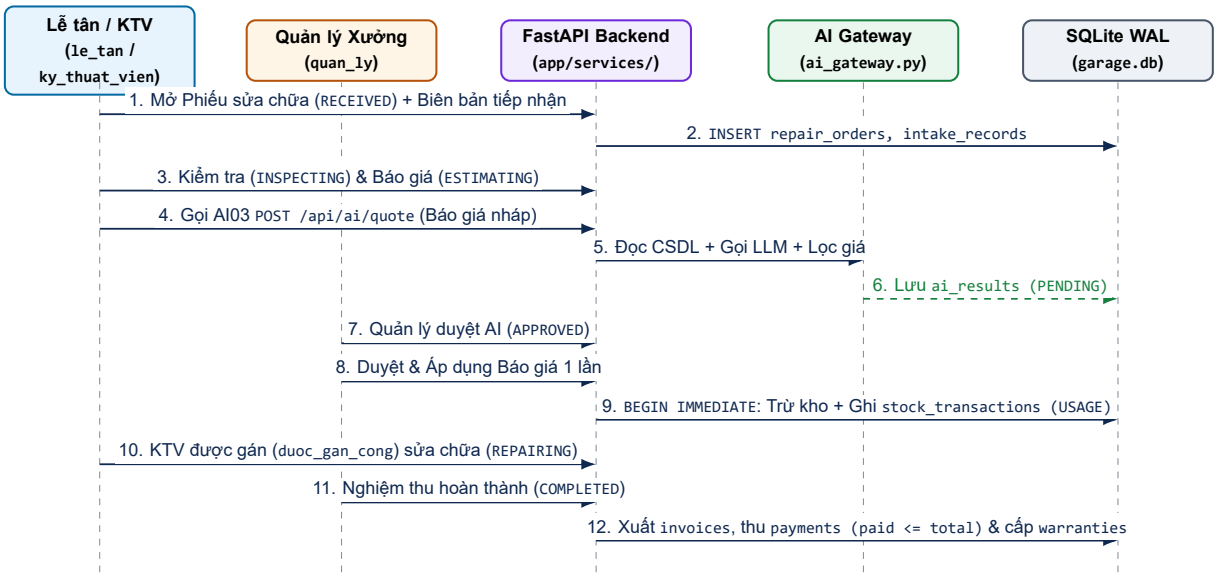
- **NFR01 (An toàn xác thực, chống CSRF và phân quyền chặt chẽ)**: Không lưu mật khẩu thô; tự động hủy phiên cũ qua `auth_version`; kiểm tra CSRF trên mọi thao tác ghi; cô lập Kỹ thuật viên theo phiếu được phân công.
- **NFR02 (Chính xác tuyệt đối về tiền tệ và tồn kho)**: Mọi cột tiền dùng kiểu số nguyên `INTEGER` (tính bằng đồng VNĐ); `unit_price` được chốt cứng tại thời điểm thêm hạng mục để việc đổi giá danh mục không làm sai lệch hóa đơn cũ; đối soát sổ cái kho luôn đạt chênh lệch bằng 0.
- **NFR03 (Hiệu năng và thời gian phản hồi)**: Các thao tác CRUD nội bộ phản hồi dưới 50ms; thời gian gọi mô hình thật trên Groq (`openai/gpt-oss-120b`) trung bình từ 2,46s đến 3,99s, đáp ứng tốt yêu cầu dưới 10 giây (tiêu chí CK-07).
- **NFR04 (Kiểm soát AI không tự quyết định tiền)**: Kết quả AI chỉ dừng ở văn bản tư vấn trong bảng `ai_results`, tuyệt đối không tự chèn hạng mục vào phiếu sửa chữa hay tự tạo hóa đơn.
- **NFR05 (Chịu lỗi và giới hạn tần suất gọi API)**: Bắt toàn bộ ngoại lệ mạng, quá thời gian chờ (`AI_TIMEOUT = 15s`), lỗi HTTP 429/500 từ nhà cung cấp AI để tự động chuyển sang `_goi_mock` mà không làm dừng ứng dụng; giới hạn tốc độ bấm nút chạy AI

(`AI_RATE_LIMIT = 10` lần / 60 giây).

- **NFR06 (Tự động nâng cấp CSDL và Sao lưu / Khôi phục an toàn)**: Tự động chạy các bản nâng cấp lược đồ `001` → `005` (`database/migrations/run_missing.py`) một cách *idempotent* khi khởi động; hỗ trợ sao lưu trực tuyến và khôi phục kèm kiểm tra `PRAGMA integrity_check`.

3.4 Kiểm soát phân quyền RBAC và Cô lập phiếu theo Kỹ thuật viên (duoc_gan_cong)

Hình 3.2 minh họa trình tự phối hợp giữa Lễ tân, Kỹ thuật viên, Quản lý, Backend FastAPI, AI Gateway và SQLite từ lúc tiếp nhận xe đến khi nghiệm thu, thanh toán và cấp bảo hành.



Hình 3.2: Biểu đồ tuần tự (Sequence Diagram) quy trình sửa chữa khép kín và duyệt kết quả AI

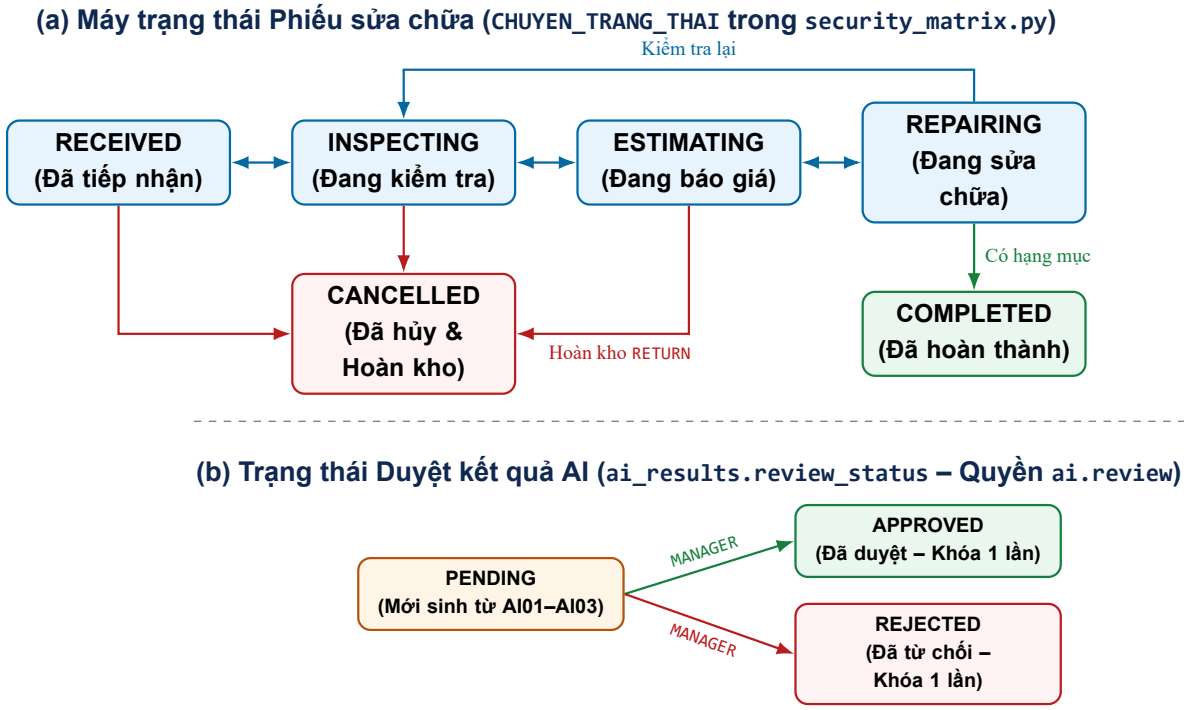
Toàn bộ ma trận phân quyền của hệ thống được định nghĩa tập trung tại một tệp duy nhất là `app/services/security_matrix.py` (biến `PERMISSIONS`) để dùng chung cho cả Backend lẫn giao diện. Đối với các thao tác sửa đổi trên Phiếu sửa chữa, hàm `_kiem_tra_quyen_thao_tac` trong `app/services/orders.py` kết hợp kiểm tra vai trò và phạm vi phân công:

$$\begin{aligned}\text{DuocThaoTac}(\text{user}, \text{phieu}) \iff & (\text{user.role} \in \{\texttt{'MANAGER'}, \texttt{'RECEPTIONIST'}\}) \\ & \vee (\text{user.role} = \texttt{'TECHNICIAN'} \wedge \text{phieu.technician_id} = \text{user.id})\end{aligned} \tag{3.1}$$

Nhờ chốt chặn này, một Kỹ thuật viên chỉ có thể cập nhật trạng thái, thêm/gỡ hạng mục, ghi nhật ký kỹ thuật hoặc chạy AI03 trên đúng những phiếu sửa chữa được giao cho mình; mọi nỗ lực can thiệp vào phiếu của Kỹ thuật viên khác hoặc phiếu chưa phân công đều bị từ chối ngay tại tầng dịch vụ.

3.5 Máy trạng thái Phiếu sửa chữa (6 trạng thái) và Quy trình Human Review

Vòng đời của một Phiếu sửa chữa (`repair_orders.status`) và trạng thái duyệt kết quả AI (`ai_results.review_status`) được quy định chặt chẽ tại biến `CHUYEN_TRANG_THAI` trong `app/services/security_matrix.py` và thể hiện trên Hình 3.3.



Hình 3.3: Máy trạng thái vòng đời Phiếu sửa chữa (6 trạng thái) và Trạng thái duyệt kết quả AI

Bốn quy tắc bảo vệ trạng thái quan trọng được cài đặt trong `app/services/orders.py` và `app/services/ai.py` gồm:

1. **Lộ trình chuyển trạng thái chống nhảy cóc (`CHUYEN_TRANG_THAI`)**:
   - Từ `RECEIVED` chỉ được chuyển sang `INSPECTING` hoặc `CANCELLED`.
   - Từ `INSPECTING` được chuyển sang `ESTIMATING`, quay lùi về `RECEIVED` hoặc hủy (`CANCELLED`).
   - Từ `ESTIMATING` được chuyển sang `REPAIRING`, quay lại `INSPECTING` hoặc hủy (`CANCELLED`).
   - Từ `REPAIRING` được chuyển sang `COMPLETED` (khi đã có ít nhất 1 hạng mục), quay lại `ESTIMATING`/`INSPECTING` khi phát sinh hạng mục hoặc hủy (`CANCELLED`) nếu chưa lập hóa đơn.
   - Hai trạng thái `COMPLETED` và `CANCELLED` là trạng thái đóng (`[]`), đồng thời khóa hoàn toàn việc thêm/gỡ hạng mục (`TRANG_THAI_KHOA_HANG_MUC`).
2. **Khóa hạng mục khi đã phát hành hóa đơn**: Hàm `_khoa_hang_muc` kiểm tra nếu phiếu đã có bản ghi trong bảng `invoices` thì chặn mọi thao tác thêm hoặc gỡ hạng mục để tổng tiền hạng mục và tổng tiền hóa đơn không bao giờ lệch nhau.
3. **Tự động hoàn trả phụ tùng về kho khi gỡ hạng mục hoặc hủy phiếu**: Khi gỡ một dòng phụ tùng hoặc chuyển phiếu sang `CANCELLED`, hệ thống tự động gọi

`ghi_giao_dich(..., +qty, 'RETURN')` trong cùng một giao dịch `BEGIN IMMEDIATE` để cộng trả lại `parts.stock` và ghi dòng đối ứng vào `stock_transactions`.

4. **Duyệt kết quả AI đúng một lần (Human Review)**: Kết quả AI sinh ra luôn ở trạng thái `PENDING`; chỉ tài khoản có quyền `ai.review` (`MANAGER`) mới được chuyển sang `APPROVED` hoặc `REJECTED`, và khi đã duyệt thì hệ thống chặn duyệt lại lần thứ hai.

3.6 Kết luận chương

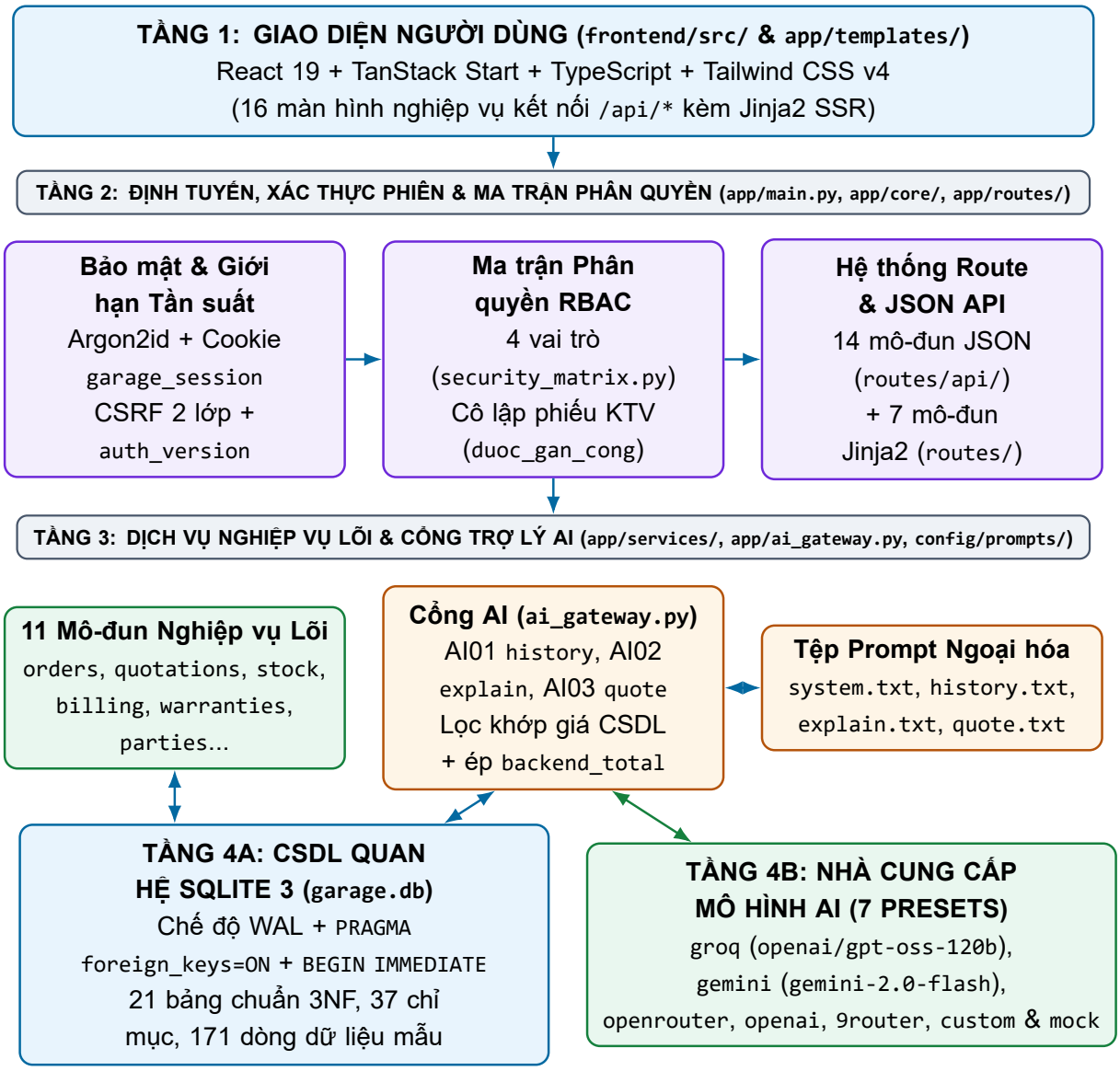
Chương 3 đã đặc tả đầy đủ 4 vai trò nhân sự, 14 yêu cầu chức năng, 6 yêu cầu phi chức năng, cơ chế cô lập phiếu của Kỹ thuật viên (`duoc_gan_cong`) và máy trạng thái 6 bước của Phiếu sửa chữa. Đây là cơ sở trực tiếp cho thiết kế kiến trúc và cơ sở dữ liệu ở Chương 4.

CHƯƠNG 4

THIẾT KẾ KIẾN TRÚC, DỮ LIỆU VÀ TÍCH HỢP AI

4.1 Kiến trúc tổng thể 4 tầng (C4 Container)

Hệ thống **GarageTools AI** được tổ chức theo kiến trúc 4 tầng tách biệt rõ trách nhiệm: (1) Tầng Giao diện người dùng (`frontend/src/` chạy React 19 + TanStack Start song song cùng giao diện Jinja2 tại `app/templates/`); (2) Tầng Định tuyến & Bảo mật (`app/main.py`, `app/core/`, `app/routes/`); (3) Tầng Dịch vụ Nghiệp vụ & AI Gateway (`app/services/`, `app/ai_gateway.py`, `config/prompts/`); và (4) Tầng Cơ sở dữ liệu quan hệ SQLite WAL (`garage.db`). Hình 4.1 minh họa sơ đồ kiến trúc thành phần chứa của hệ thống.

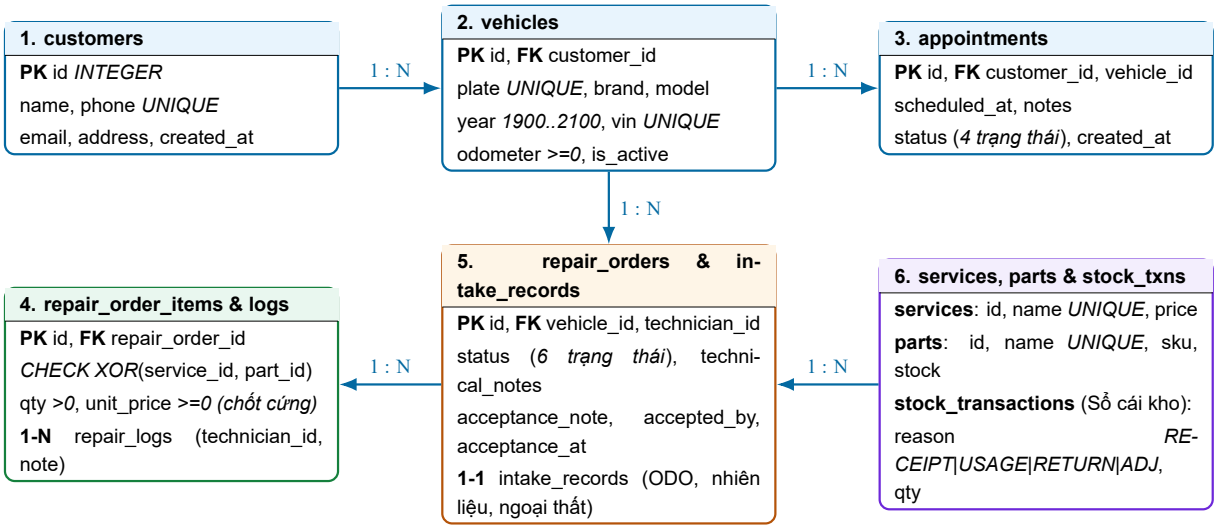


Hình 4.1: Sơ đồ kiến trúc thành phần chứa (C4 Container) của hệ thống GarageTools AI

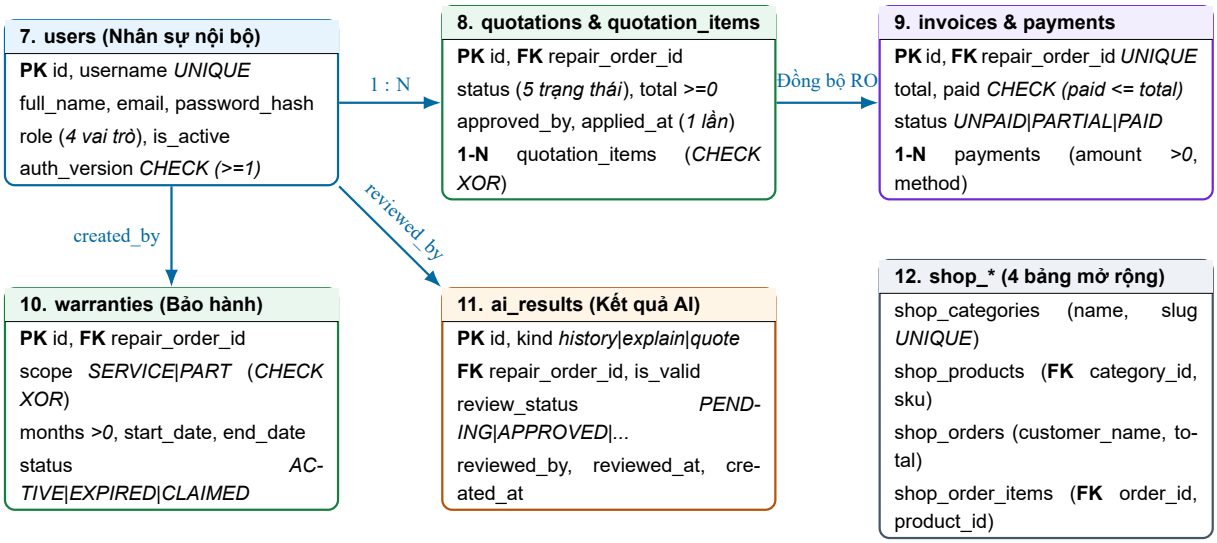
Mọi yêu cầu từ trình duyệt đều đi qua lớp kiểm tra phiên đăng nhập (`garage_session`), đối chiếu `auth_version`, kiểm tra CSRF token (`garage_csrf` / `X-CSRF-Token`) và kiểm tra quyền trong `security_matrix.py` trước khi thực thi logic tại `app/services/`.

4.2 Thiết kế cơ sở dữ liệu quan hệ (ERD 21 bảng chuẩn 3NF)

Cơ sở dữ liệu của hệ thống gồm **21 bảng** (17 bảng nghiệp vụ lõi của garage và 4 bảng mở rộng cửa hàng dụng cụ `shop_*`) cùng **37 chỉ mục (indexes)**, được định nghĩa tại `database/schema.sql` và tự động nâng cấp qua 5 bản di trú (`001` → `005`) trong `database/migrations/`. Sơ đồ ERD được chia thành hai nhóm: Phân hệ Tiếp nhận, Phiếu sửa chữa & Sổ cái Kho (Hình 4.2) và Phân hệ Người dùng, Báo giá, Hóa đơn, Bảo hành & Kết quả AI (Hình 4.3).



Hình 4.2: Sơ đồ thực thể liên kết (ERD) phân hệ Khách hàng, Phương tiện, Phiếu sửa chữa và Sổ cái Kho



Hình 4.3: Sơ đồ thực thể liên kết (ERD) phân hệ Người dùng, Báo giá, Hóa đơn, Bảo hành và Kết quả AI

Bảng 4.1 đặc tả chính xác tên bảng, khóa chính, khóa ngoại và các ràng buộc toàn vẹn (`CHECK`, `UNIQUE`, `ON DELETE RESTRICT`) đúng như trong tệp `database/schema.sql`.

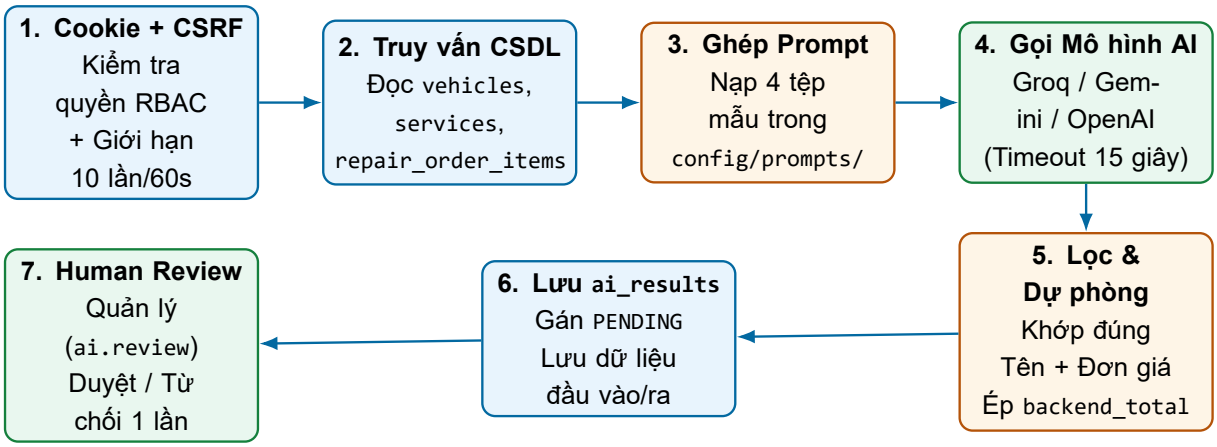
Bảng 4.1: Đặc tả cấu trúc 21 bảng dữ liệu và ràng buộc toàn vẹn trong `database/schema.sql`

Tên bảng	**Khóa chính & Khóa ngoại**	**Vai trò nghiệp vụ và Ràng buộc toàn vẹn (Constraints)**
1. `users`	PK: `id`	Tài khoản nhân viên; `username UNIQUE`; `role IN ('MANAGER', 'RECEPTIONIST', 'TECHNICIAN', 'CASHIER')`; `is_active IN (0,1)`; `auth_version >= 1`.
2. `customers`	PK: `id`	Hồ sơ khách hàng; lưu `name`, `phone UNIQUE`, `email`, `address`, `created_at`.
3. `vehicles`	PK: `id`; FK: `customer_id`	Hồ sơ phương tiện; `plate UNIQUE`, `vin UNIQUE`; `year BETWEEN 1900 AND 2100`; `odometer >= 0`; `is_active IN (0,1)`.
4. `appointments`	PK: `id`; FK: `customer_id`, `vehicle_id`	Lịch hẹn dịch vụ; giữ cả `customer_id` và `vehicle_id` để đặt lịch nhanh; `status IN ('SCHEDULED', 'CONFIRMED', 'CANCELLED', 'DONE')`.
5. `services`	PK: `id`	Danh mục dịch vụ sửa chữa; `name UNIQUE`, `description` (cho phép `NULL`), `price >= 0` (kiểu `INTEGER`), `is_active`.
6. `parts`	PK: `id`	Danh mục phụ tùng; `name UNIQUE`, `sku`, `stock >= 0`, `min_stock >= 0`, `price >= 0`, `is_active`.
7. `stock_transactions`	PK: `id`; FK: `part_id`, `repair_order_id`, `created_by`	Sổ cái tồn kho bất biến; `reason IN ('RECEIPT', 'USAGE', 'RETURN', 'ADJUSTMENT')`; duy trì bất biến `parts.stock` $= \sum$ `qty`.
8. `repair_orders`	PK: `id`; FK: `vehicle_id`, `technician_id`, `accepted_by`	Phiếu sửa chữa; 6 trạng thái (`RECEIVED`, `INSPECTING`, `ESTIMATING`, `REPAIRING`, `COMPLETED`, `CANCELLED`); `CHECK (acceptance_at IS NULL OR accepted_by IS NOT NULL)`.
9. `repair_order_items`	PK: `id`; FK: `repair_order_id`, `service_id`, `part_id`	Hạng mục của phiếu; **ràng buộc XOR**: `(service_id IS NOT NULL AND part_id IS NULL) OR (service_id IS NULL AND part_id IS NOT NULL)`; `qty > 0`, `unit_price >= 0` (chốt cứng).

Tên bảng	**Khóa chính & Khóa ngoại**	**Vai trò nghiệp vụ và Ràng buộc toàn vẹn (Constraints)**
10. `repair_logs`	PK: `id`; FK: `repair_order_id`, `technician_id`	Nhật ký kỹ thuật ghi lại từng bước kiểm tra, sửa chữa và chuyển trạng thái trên phiếu.
11. `intake_records`	PK: `id`; FK: `repair_order_id` `UNIQUE`, `created_by`	Biên bản tiếp nhận xe (1–1 với phiếu sửa chữa); lưu `odometer_at_intake >= 0`, `fuel_level`, `exterior_condition`, `accessories`.
12. `quotations`	PK: `id`; FK: `repair_order_id`, `created_by`, `approved_by`, `applied_by`	Báo giá chính thức; 5 trạng thái (`DRAFT`, `SENT`, `APPROVED`, `REJECTED`, `EXPIRED`); cột `applied_at` đảm bảo chỉ áp dụng 1 lần (*idempotent*).
13. `quotation_items`	PK: `id`; FK: `quotation_id`, `service_id`, `part_id`	Hạng mục trong báo giá; áp dụng cùng **ràng buộc XOR** giữa dịch vụ và phụ tùng; `qty > 0`, `unit_price >= 0`.
14. `invoices`	PK: `id`; FK: `repair_order_id` `UNIQUE`	Hóa đơn của phiếu (1–1); `status IN ('UNPAID','PARTIAL','PAID')`; **ràng buộc chống thu vượt**: `CHECK (paid <= total)`.
15. `payments`	PK: `id`; FK: `invoice_id`	Lịch sử các đợt thanh toán; `amount > 0`; `method IN ('CASH','CARD','TRANSFER','ZALO')`.
16. `ai_results`	PK: `id`; FK: `repair_order_id`, `reviewed_by`	Lưu kết quả AI và trạng thái duyệt; `kind IN ('history', 'explain', 'quote')`; `review_status IN ('PENDING', 'APPROVED', 'REJECTED')`; `CHECK (review_status = 'PENDING' OR (reviewed_by IS NOT NULL AND reviewed_at IS NOT NULL))`.
17. `warranties`	PK: `id`; FK: `repair_order_id`, `service_id`, `part_id`, `created_by`	Bảo hành theo hạng mục của phiếu đã hoàn thành; `scope IN ('SERVICE', 'PART')` kèm **ràng buộc XOR**; `months > 0`, `end_date > start_date`.
18–21. `shop_categories`, `shop_products`, `shop_orders`, `shop_order_items`	PK: `id`; FK: `category_id`, `order_id`, `product_id`	Bốn bảng mở rộng cho mô-đun cửa hàng dụng cụ/phụ kiện (`shop_orders.status IN ('PENDING', 'PAID', 'SHIPPED', 'CANCELLED')`).

4.3 Thiết kế Pipeline AI 7 bước và Cơ chế Chống ảo giác giá

Để đảm bảo AI hoạt động đúng vai trò trợ lý tư vấn, hệ thống thiết kế luồng xử lý 7 bước tại `app/ai_gateway.py` và `app/services/ai.py` (Hình 4.4) cùng bảng kiểm soát dữ liệu cho từng chức năng (Bảng 4.2).



Hình 4.4: Quy trình xử lý 7 bước của AI Gateway, lọc khớp giá CSDL và duyệt Human Review

Bảng 4.2: Dữ liệu đầu vào và cơ chế kiểm soát đầu ra của 3 chức năng AI

Chức năng AI	**Biến dữ liệu ghép vào Prompt (`config/prompts/*.txt`)**	**Ràng buộc đầu ra & Xử lý ca biên tại `ai_gateway.py`**
AI01: Tóm tắt lịch sử sửa chữa (`history.txt`)	Biến `{{repair_history}}`: thông tin biển số, hãng, dòng, năm sản xuất và danh sách các phiếu sửa chữa + hạng mục trước đó của xe.	Trả về đúng 4 mục đánh số `1..4`; nếu xe chưa từng sửa chữa (`orders` rỗng, ví dụ `vehicle_id=6`), ghi rõ "Chưa có lịch sử sửa chữa" và cấm suy đoán lỗi.
AI02: Giải thích dịch vụ (`explain.txt`)	Biến `{{service}}`: tên dịch vụ, đơn giá niêm yết (`services.price`) và mô tả kỹ thuật (`services.description`).	Trả về 3 mục dễ hiểu cho khách hàng; nếu dịch vụ chưa có mô tả (`description IS NULL`, ví dụ `service_id=7`) thì tự điền câu mặc định ở mục 1, không báo lỗi 500.
AI03: Báo giá nháp (`quote.txt`)	Biến `{{items}}` và `{{backend_total}}`: danh sách hạng mục từ `repair_order_items` (hoặc từ báo giá `quotations` đang mở) và tổng tiền do Backend tự cộng.	Chỉ giữ lại các dòng trong JSON khớp chính xác (`name, qty, unit_price`) với CSDL; luôn gán `total = backend_total`; tự động fallback sang `_goi_mock` nếu lỗi.

4.4 Ba vòng thử nghiệm và tối ưu Prompt (`KT3_PROMPT_EXPERIMENTS.md`)

Toàn bộ lời nhắc gửi lên mô hình được tách khỏi mã nguồn Python thành 4 tệp văn bản trong `config/prompts/` (`system.txt`, `history.txt`, `explain.txt`, `quote.txt`). Nhóm đã thực hiện 3 vòng thử nghiệm và cải tiến cho từng chức năng:

- **Vòng 1 – Câu lệnh ngắn, không quy định cấu trúc đầu ra**: Ví dụ với AI01 chỉ viết *"Tóm tắt lịch sử sửa chữa sau cho nhân viên lễ tân: {{repair_history}}"*; với AI03 chỉ yêu cầu *"Tạo báo giá nháp dạng JSON từ danh sách hạng mục"*. **Vấn đề gặp phải**: Mỗi lần gọi AI01 trả về một bố cục khác nhau, và khi gặp xe mới chưa có lịch sử thì mô hình tự suy diễn các lỗi thường gặp của dòng xe; với AI03, mô hình trả kèm khối ```` ```json ```` và có lần cộng nhầm tổng tiền `total`.
- **Vòng 2 – Quy định rõ bố cục mục và khóa JSON, truyền sẵn `backend_total`**: Quy định cố định 4 mục cho AI01, 3 mục cho AI02 và truyền sẵn biến `{{backend_total}}` vào `quote.txt`. **Kết quả & Điểm còn thiếu**: Đầu ra đã đi vào khuôn khổ, nhưng khi kiểm thử với dịch vụ có `description IS NULL` (dịch vụ `#7` cân chỉnh thước lái) ở AI02 thì mô hình giải thích chung chung và thiếu đơn giá niêm yết; với AI03 khi phiếu sửa chữa trống hạng mục thì chưa có quy định trả mảng rỗng rõ ràng.
- **Vòng 3 – Bổ sung quy tắc xử lý dữ liệu trống và bộ lọc hậu kiểm ở Backend**: Hoàn thiện cả 4 tệp prompt trong `config/prompts/`: cấm suy đoán khi xe chưa có lịch sử, truyền kèm đơn giá niêm yết và quy định câu trả lời mặc định khi `description` trống cho AI02, đồng thời bổ sung bộ lọc đối chiếu từng dòng `(name, qty, unit_price)` và cưỡng bức `total = backend_total` trong hàm `bao_gia_nhap`.

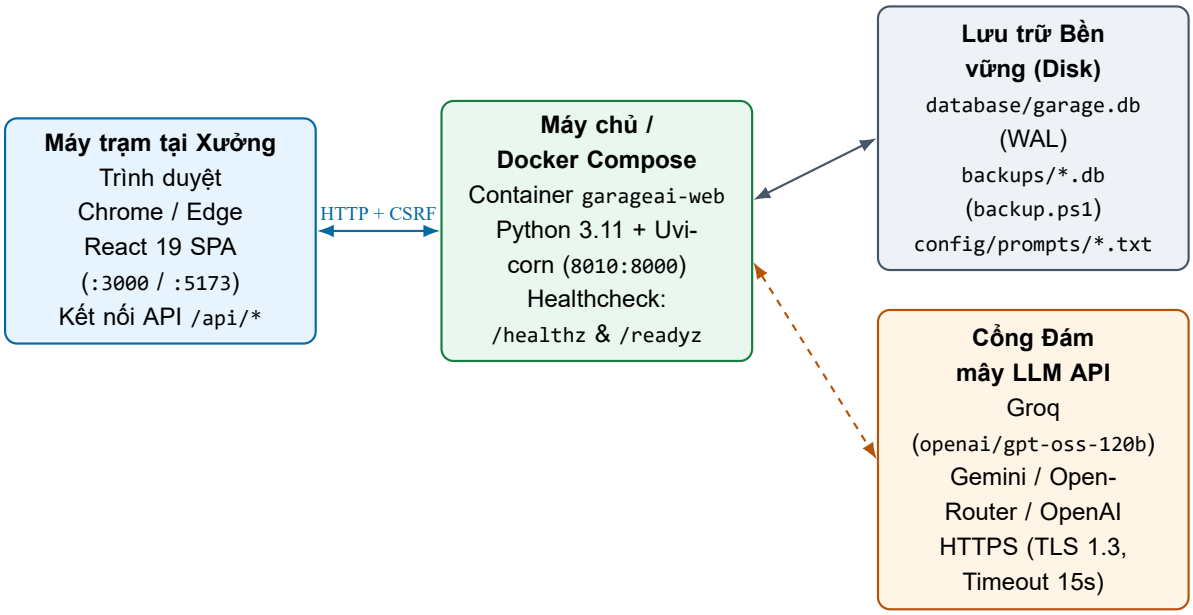
```
# File: config/prompts/system.txt
Ban la tro ly AI noi bo cua garage sua chua o to. Nguyen tac bat buoc:
1. Chi su dung thong tin co trong du lieu duoc cung cap. Khong suy doan, khong bia
   them hang muc, gia tien, ngay thang hay nguyen nhan hong hoc.
2. Neu du lieu thieu hoac trong, noi ro la chua co thong tin; khong tu dien.
3. Khong dua ra chan doan ky thuat thay cho ky thuat vien.
4. Tra loi bang tieng Viet ro rang, ngan gon, dung trong tam.

# File: config/prompts/quote.txt
Lap bao gia nhap tu danh sach hang muc da duoc ghi nhan trong phieu sua chua.
Quy tac bat buoc:
- Chi dung cac hang muc co trong danh sach. Khong them hang muc moi, khong doi
  so luong, khong doi don gia.
- Giu nguyen tong tien do he thong tinh san: {{backend_total}} VND.
- Neu danh sach hang muc trong, tra ve "items": [] va "total": 0...
```

Listing 4.1: Nội dung thực tế của tệp config/prompts/system.txt và config/prompts/quote.txt

4.5 Thiết kế kiến trúc triển khai và Hệ thống Route / API

Hệ thống hỗ trợ chạy trực tiếp bằng Uvicorn hoặc đóng gói qua Docker Compose (`Dockerfile`, `docker-compose.yml`) ánh xạ cổng `8010:8000` kèm kiểm tra sức khỏe tại `/healthz` và `/readyz` (Hình 4.5). Bảng 4.3 liệt kê các nhóm API chính trong `app/routes/api/`.



Hình 4.5: Sơ đồ triển khai hệ thống trên máy trạm tại xưởng hoặc Docker Compose

Bảng 4.3: Danh mục các mô-đun JSON API chính trong `app/routes/api/`

Mô-đun & Đường dẫn	Phương thức	Quyền kiểm tra	Chức năng nghiệp vụ chính
`auth_api.py` (`/api/auth/*`)	POST, GET	Công khai / Đã đăng nhập	Đăng nhập Argon2id, cấp cookie `garage_session` + `garage_csrf`, đổi mật khẩu (tăng `auth_version`), đăng xuất.
`parties_api.py` (`/api/customers`, `/vehicles`, `/appointments`)	GET, POST, PUT, DELETE	`customer.*`, `vehicle.*`, `appointment.*`	CRUD khách hàng, phương tiện, lịch hẹn; tìm kiếm, lọc và phân trang.
`orders_api.py` (`/api/orders/*`)	GET, POST, DELETE	`order.*` + `duoc_gan_cong`	Tạo phiếu sửa chữa, lưu biên bản tiếp nhận (`intake_records`), phân công KTV, thêm/gỡ hạng mục, đổi trạng thái, nghiệm thu.
`quotations_api.py` (`/api/quotations/*`)	GET, POST, DELETE	`quote.manage`, `quote.approve`	Tạo báo giá, thêm/gỡ dòng báo giá, gửi/duyệt báo giá và áp dụng *idempotent* vào phiếu sửa chữa.
`catalog_api.py` & `stock_api.py`	GET, POST, PUT	`catalog.*`, `stock.manage`	Quản lý danh mục dịch vụ, phụ tùng, nhập/điều chỉnh kho qua sổ cái `stock_transactions` và đối soát tồn kho.
`billing_api.py` (`/api/invoices`, `/payments`)	GET, POST, DELETE	`invoice.manage`, `payment.record`	Lập hóa đơn cho phiếu đã `COMPLETED`, thu tiền từng phần/toàn bộ (`paid <= total`) và hủy khoản thu nhầm.
`warranties_api.py` (`/api/warranties/*`)	GET, POST	`warranty.manage`	Tạo bảo hành theo hạng mục của phiếu đã hoàn thành, yêu cầu bảo hành (`CLAIMED`) hoặc hủy bảo hành.
`ai_api.py` (`/api/ai/*`)	GET, POST	`ai.history`, `ai.explain`, `ai.quote`, `ai.review`	Chạy 3 chức năng AI01–AI03 và duyệt/từ chối kết quả AI trong bảng `ai_results`.
`dashboard_api.py`, `reports_api.py`, `admin_api.py`	GET, POST, PUT	`dashboard.view`, `report.view`, `user.*`	Tổng quan garage, 5 báo cáo thống kê và quản trị tài khoản nhân viên.

4.6 Kết luận chương

Chương 4 đã trình bày đầy đủ thiết kế kiến trúc 4 tầng, lược đồ CSDL 21 bảng chuẩn 3NF với 37 chỉ mục, quy trình xử lý 7 bước của AI Gateway và 3 vòng thử nghiệm cải tiến Prompt. Chương 5 tiếp theo sẽ trình bày chi tiết việc hiện thực hóa các thiết kế này trong mã nguồn.

CHƯƠNG 5

HIỆN THỰC HÓA HỆ THỐNG

5.1 Tổng quan cấu trúc mã nguồn thực tế

Mã nguồn của hệ thống được tổ chức thành hai nhánh chính tại thư mục gốc của kho lưu trữ: `GarageAI_Project/` (chứa Backend FastAPI, CSDL SQLite, tệp cấu hình Prompt, giao diện Jinja2 và bộ kiểm thử Python) và `frontend/` (chứa ứng dụng giao diện React 19 + TanStack Start), cùng thư mục `scripts/` chứa các kịch bản kiểm thử và sao lưu tự động. Bảng 5.1 mô tả chính xác các tệp mã nguồn của dự án.

Bảng 5.1: Cấu trúc phân bổ mã nguồn thực tế của dự án `Nhom23_GarageToolsAI`

Đường dẫn mã nguồn	**Trách nhiệm và thành phần hiện thực hóa**	**Công nghệ sử dụng**
`GarageAI_Project/app/main.py`, `models.py`, `crud.py`	Khởi tạo ứng dụng (`create_app`), tự động chạy migrations, đăng ký middleware bảo mật, định nghĩa Pydantic v2 models và tầng truy vấn tham số hóa.	Python 3.12+, FastAPI, Pydantic v2
`GarageAI_Project/app/core/*.py`	Gồm `config.py` (7 presets AI), `db.py` (kết nối WAL & `transaction()` dùng `BEGIN IMMEDIATE`), `security.py` (Argon2id, HMAC cookie, CSRF), `ratelimit.py`, `money.py`, `pagination.py`, `errors.py`.	SQLite 3, Argon2-cffi, HMAC-SHA256
`GarageAI_Project/app/services/*.py`	11 mô-đun nghiệp vụ thuần: `auth.py`, `security_matrix.py`, `orders.py`, `quotations.py`, `stock.py`, `warranties.py`, `billing.py`, `parties.py`, `catalog.py`, `ai.py`, `reports.py`.	Tầng nghiệp vụ nguyên tử
`GarageAI_Project/app/ai_gateway.py` & `config/prompts/*.txt`	Lớp gọi mô hình ngôn ngữ (`mock`, `groq`, `openrouter`, `gemini`, `openai`, `9router`, `custom`), nạp 4 tệp prompt (`system`, `history`, `explain`, `quote`), lọc khớp giá và fallback `_goi_mock`.	HTTPX, JSON Parser, Price Filter
`GarageAI_Project/app/routes/*.py` & `routes/api/*.py`	7 mô-đun route HTML Jinja2 (`auth`, `orders`, `parties`, `quotations`, `stock`, `system`, `warranties`) và 14 mô-đun JSON API trong `routes/api/`.	FastAPI APIRouter, Jinja2
`GarageAI_Project/database/`	Gồm `schema.sql` (21 bảng, 37 chỉ mục), `seed_data.sql` (171 dòng mẫu), `init_db.py` và `migrations/` (`001` → `005` + `run_missing.py`).	SQLite 3 DDL, Auto-Migrate
`frontend/src/`	Ứng dụng giao diện chính gồm 16 màn hình trong `src/routes/`, `src/lib/rbac.tsx`, `src/services/api.ts` và kiểm thử `vitest`.	React 19, TanStack Start, Tailwind v4
`GarageAI_Project/tests/` & `scripts/*.ps1`	Bộ 231 ca kiểm thử Python (`run_all.py`), 28 ca kiểm thử Frontend (`vitest`) và 4 kịch bản PowerShell (`smoke.ps1`, `e2e.ps1`, `backup.ps1`, `restore.ps1`).	Python `unittest`, Vitest, PowerShell

5.2 Hiện thực hóa các chốt chặn nghiệp vụ và bảo mật ở Backend

5.2.1 Ma trận Phân quyền tập trung và Cô lập phiếu theo Kỹ thuật viên

Trong `app/services/security_matrix.py`, biến `PERMISSIONS` ánh xạ từng quyền cụ thể tới tập hợp các vai trò được phép, kết hợp với hàm `duoc_gan_cong(user, phieu)` để kiểm soát phạm vi thao tác trên từng phiếu sửa chữa. Bảng 5.2 tóm tắt cách hệ thống phân giải quyền cho 4 vai trò.

Bảng 5.2: Ma trận phân giải quyền hạn theo 4 vai trò trong `app/services/security_matrix.py`

Phân hệ	**Thao tác**	**Quản lý (MANAGER)**	**Lễ tân (RECEPTIONIST)**	**Kỹ thuật viên (TECHNICIAN)**	**Thu ngân (CASHIER)**
Phiếu sửa chữa	Xem danh sách / Chi tiết	Toàn bộ phiếu	Toàn bộ phiếu	Chỉ phiếu được giao (`duoc_gan_cong`)	Toàn bộ phiếu (Chỉ xem)
	Tạo phiếu / Giao KTV / Nghiệm thu	Cho phép	Cho phép	Từ chối (HTTP 403)	Từ chối (HTTP 403)
	Đổi trạng thái / Sửa ghi chú / Ghi log	Cho phép	Cho phép điều phối trạng thái; chặn thêm hạng mục	Chỉ trên phiếu có `technician_id == user.id`	Từ chối (HTTP 403)
Báo giá	Tạo & Gửi báo giá (`quote.manage`)	Cho phép	Cho phép	Từ chối (HTTP 403)	Từ chối (HTTP 403)
	Duyệt & Áp dụng (`quote.approve`)	**Duy nhất duyệt**	Chỉ được áp dụng báo giá đã duyệt	Từ chối (HTTP 403)	Từ chối (HTTP 403)
Trợ lý AI	Chạy AI01, AI02, AI03	AI01, AI02, AI03	AI01, AI02, AI03	Chỉ chạy AI03 (`ai.quote`) trên phiếu được giao	Từ chối (HTTP 403)
	Duyệt kết quả (`ai.review`)	**Duy nhất cho phép**	Từ chối (HTTP 403)	Từ chối (HTTP 403)	Từ chối (HTTP 403)
Hóa đơn & Thu	Lập hóa đơn / Thu tiền	Lập HĐ & Thu tiền	Lập HĐ khi phiếu `COMPLETED`	Từ chối (HTTP 403)	**Lập HĐ, Thu tiền & Hủy khoản thu**
Kho & Danh mục	Sửa danh mục / Nhập chỉnh kho	Toàn quyền	Nhập/Chỉnh kho (`stock.manage`)	Chỉ xem danh mục	Chỉ xem danh mục

5.2.2 Bốn chốt chặn bảo vệ Tiền tệ, Sổ cái Tồn kho và Hóa đơn

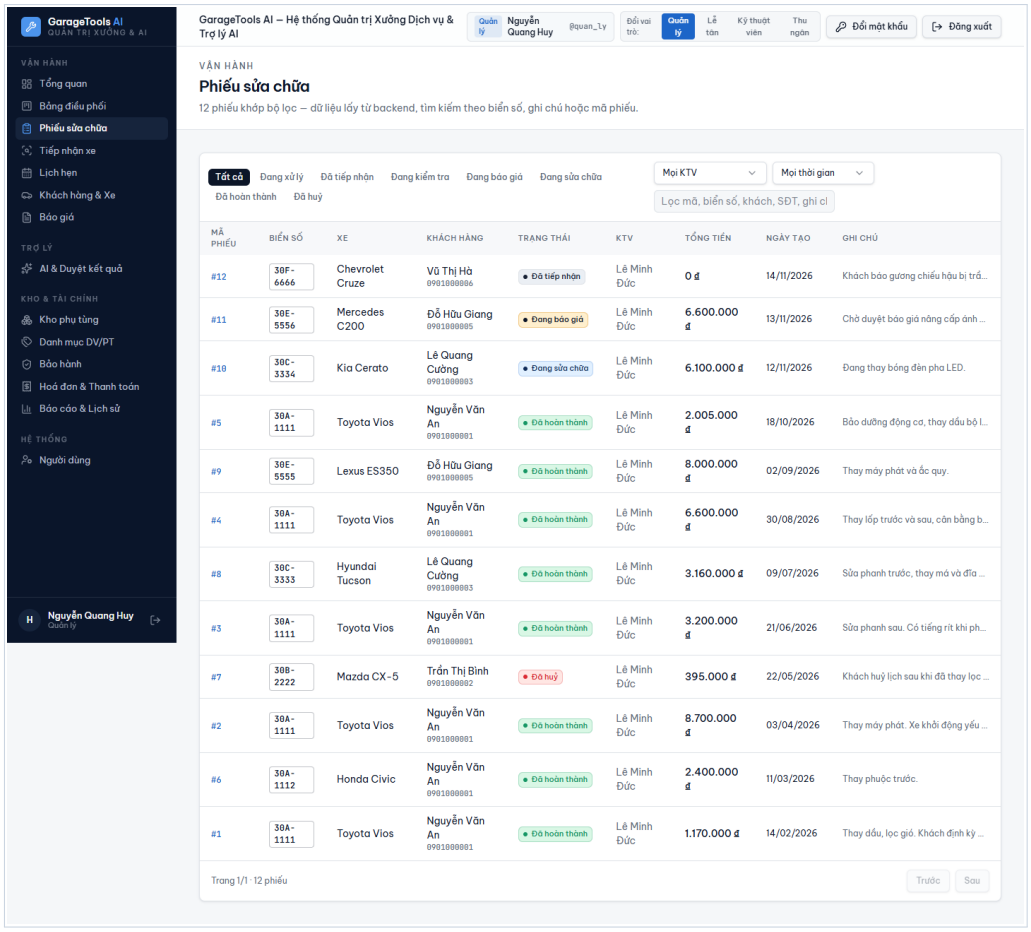
Để tránh mọi sai lệch về số tiền và tồn kho khi nhiều nhân viên thao tác cùng lúc, các mô-đun trong `app/services/` thực thi các điều kiện kiểm tra nguyên tử được tổng hợp tại Bảng 5.3.

Bảng 5.3: Các chốt chặn kỹ thuật bảo vệ toàn vẹn giao dịch trong `app/services/`

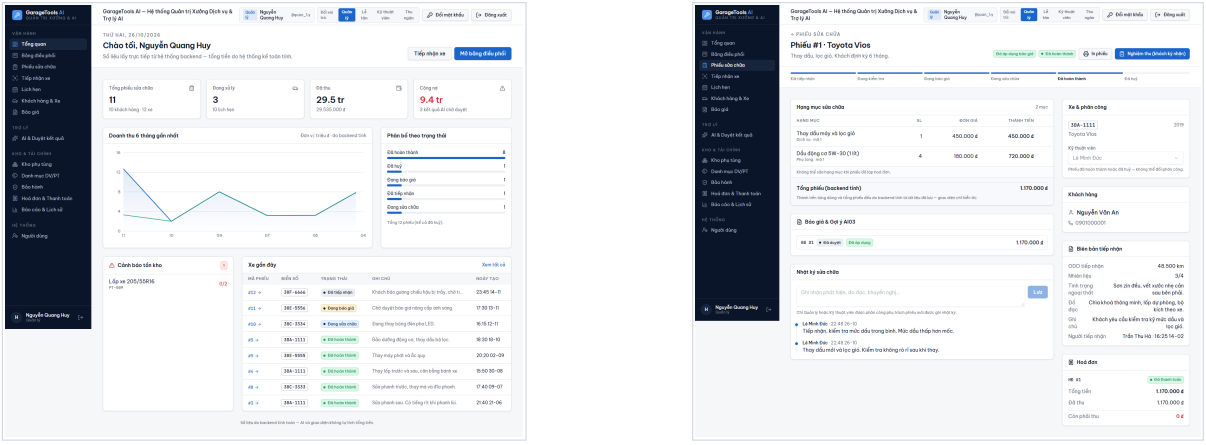
Nghiệp vụ	**Hàm xử lý chính**	**Mô-đun**	**Chốt chặn kỹ thuật trong khối `BEGIN IMMEDIATE`**	**Kết quả bảo vệ**
Trừ / Cộng tồn kho qua Sổ cái	`ghi_giao_dich`	`services/stock.py`	Thực thi câu lệnh nguyên tử `UPDATE parts SET stock = stock + ? WHERE id = ? AND stock + ? >= 0` kèm `INSERT INTO stock_transactions`.	Chặn tuyệt đối xuất âm kho; giữ $\texttt{stock} = \sum \texttt{qty}$.
Thêm hạng mục vào phiếu	`them_hang_muc`	`services/orders.py`	Kiểm tra phiếu chưa đóng và chưa có hóa đơn (`_khoa_hang_muc`); luôn đọc `price` từ CSDL và chốt cứng vào cột `unit_price`.	Đổi giá danh mục không làm thay đổi hóa đơn cũ.
Áp dụng Báo giá vào phiếu	`ap_dung_vao_phieu`	`services/quotations.py`	Yêu cầu `status == 'APPROVED'` và `applied_at IS NULL`; chèn hạng mục, trừ kho qua sổ cái và gán ngay `applied_at`.	Áp dụng *idempotent* đúng 1 lần; chặn trừ kho kép.
Nghiệm thu hoàn thành phiếu	`nghiem_thu_phieu`	`services/orders.py`	Phiếu phải ở trạng thái `COMPLETED` và `acceptance_at IS NULL`; ghi nhận `accepted_by`, `acceptance_at`, `acceptance_note`.	Chặn nghiệm thu lặp lại lần 2 (HTTP 409).
Thu tiền & Hủy khoản thu	`ghi_thanh_toan`, `huy_thanh_toan`	`services/billing.py`	Kiểm tra ngay trong SQL: `UPDATE invoices SET paid = paid + ? WHERE id = ? AND total - paid >= ?`; tự cập nhật `PARTIAL`/`PAID`.	Chặn thu vượt số tiền còn nợ (HTTP 409).
Duyệt kết quả AI	`duyet_ket_qua`	`services/ai.py`	Kiểm tra quyền `ai.review` và `review_status == 'PENDING'`; cập nhật `reviewed_by`, `reviewed_at`.	Chặn duyệt lại kết quả đã duyệt.

5.3 Hiện thực hóa Giao diện Người dùng và Ảnh minh chứng thực tế

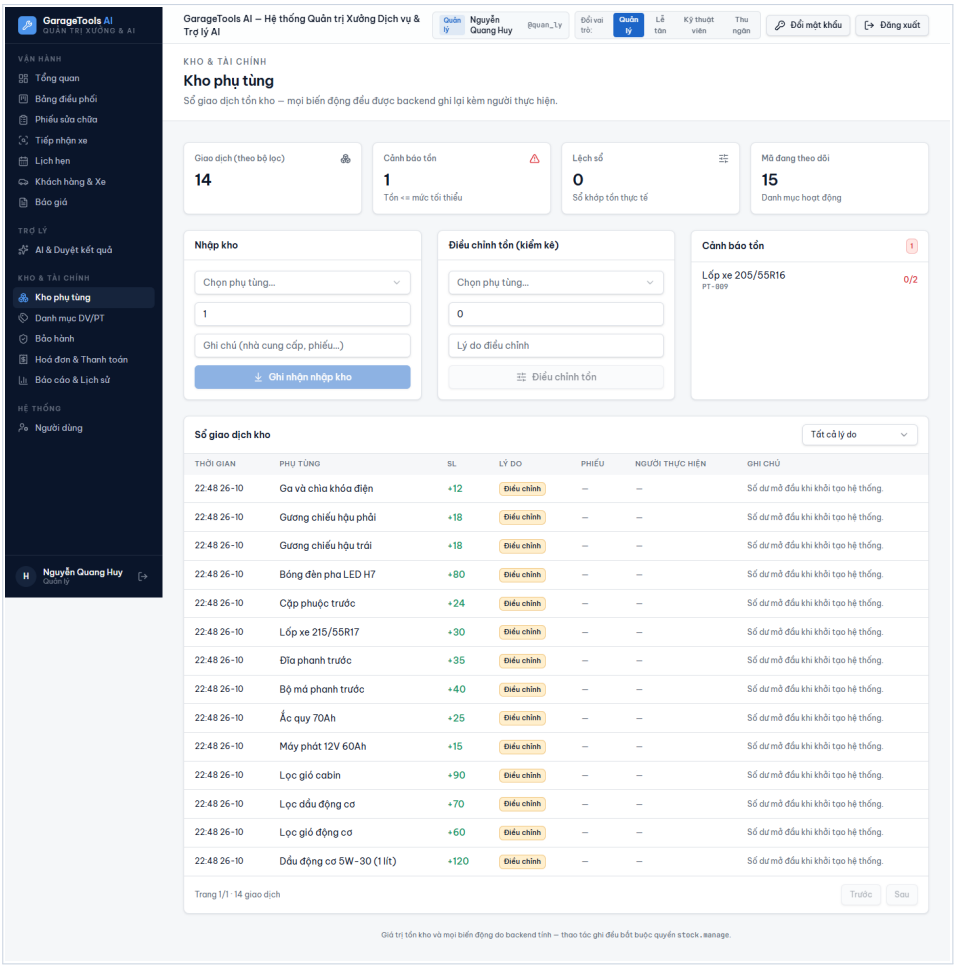
Giao diện người dùng tại `frontend/src/routes/` gồm 16 màn hình nghiệp vụ đầy đủ kết nối tới Backend qua `/api/*`. Các Hình 5.1, 5.2, 5.3 và 5.4 dưới đây là ảnh chụp màn hình vận hành thực tế của hệ thống **GarageTools AI**.



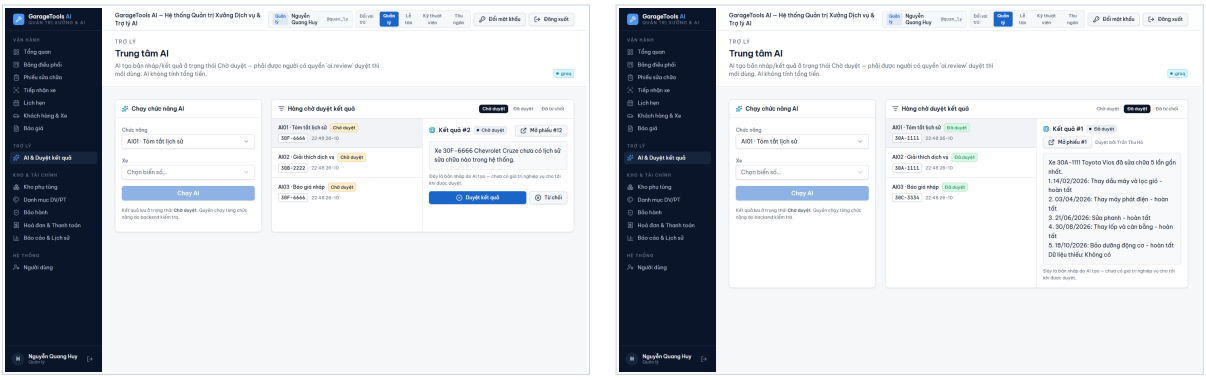
Hình 5.1: Ảnh minh chứng thực tế: Giao diện Quản lý và lọc Phiếu sửa chữa theo 6 trạng thái



Hình 5.2: Ảnh minh chứng thực tế: (Trái) Bảng điều khiển Tổng quan Dashboard; (Phải) Chi tiết Phiếu sửa chữa #1 kèm hạng mục chốt giá và Biên bản tiếp nhận xe



Hình 5.3: Ảnh minh chứng thực tế: Giao diện Kho phụ tùng và Sổ cái giao dịch đối soát thời gian thực (`Lệch sổ: 0`)



Hình 5.4: Ảnh minh chứng thực tế Trung tâm AI: (Trái) Kết quả mới sinh ở trạng thái *Chờ duyệt*; (Phải) Sau khi Quản lý phê duyệt chuyển sang *Đã duyệt*

Các điểm nổi bật trên giao diện thực tế gồm:

1. **Tổng quan Garage & Bảng điều phối Kanban chịu tải cao (30–50+ xe/ngày)**: Thống kê trực tiếp số phiếu đang mở, doanh thu thực thu, cảnh báo phụ tùng chạm mức tồn tối thiểu (`stock <= min_stock`) và bảng kéo-thả 6 trạng thái hỗ trợ cuộn độc lập từng cột, lọc theo Kỹ thuật viên, chế độ thẻ thu gọn (*Compact Mode*) và thu gọn cột kết thúc.
2. **Màn hình Tiếp nhận xe tích hợp AI01 & Chi tiết Phiếu sửa chữa tích hợp AI03**: Cho

phép chạy AI01 tóm tắt lịch sử xe ngay trước khi lập phiếu tiếp nhận, cập nhật ghi chú kỹ thuật xuyên suốt quá trình kiểm tra, chạy AI03 gợi ý giải trình báo giá ngay trong phiếu, nghiệm thu bàn giao, lập hóa đơn, cấp bảo hành và in ấn biểu mẫu (`@media print`).

3. **Màn hình Trợ lý AI & Hàng đợi Duyệt kết quả (`/ai` và `/ai-review`)**: Tách biệt rõ khu vực chạy thử nghiệm 3 chức năng AI01–AI03 và hàng đợi duyệt dành riêng cho Quản lý (`MANAGER`).

5.4 Công thức Bất biến Số học về Tiền tệ và Sổ cái Tồn kho

Toàn bộ việc tính toán tiền tệ và tồn kho trong hệ thống đều dùng số nguyên (`INTEGER`) và thỏa mãn 3 bất biến toán học được kiểm tra tự động trong bộ test:

$$\forall p \in \texttt{parts}: \quad p.\texttt{stock} = \sum_{t \in \texttt{stock_transactions}(p)} t.\texttt{qty} \geq 0 \tag{5.1}$$

$$\forall r \in \texttt{repair_orders}: \quad \text{TongPhieu}(r) = \sum_{i \in \texttt{repair_order_items}(r)} (i.\texttt{qty} \times i.\texttt{unit_price}) \tag{5.2}$$

$$\forall h \in \texttt{invoices}: \quad h.\texttt{paid} = \sum_{k \in \texttt{payments}(h)} k.\texttt{amount} \leq h.\texttt{total} \tag{5.3}$$

Đối với hàm đối soát kho (`doi_soat_so_cai` trong `app/services/stock.py` và endpoint `GET /api/stock/audit`), hệ thống so sánh trực tiếp cột `parts.stock` với tổng `SUM(stock_transactions.qty)` của từng phụ tùng; trong suốt toàn bộ các kịch bản kiểm thử E2E và UAT, danh sách lệch kho luôn rỗng tuyệt đối.

5.5 Kết luận chương

Chương 5 đã trình bày chi tiết cấu trúc mã nguồn và cách hiện thực hóa các chốt chặn bảo mật, phân quyền, giao dịch nguyên tử `BEGIN IMMEDIATE`, sổ cái tồn kho và cổng AI trong dự án. Chương 6 tiếp theo sẽ trình bày kết quả kiểm thử tự động trên toàn bộ hệ thống.

CHƯƠNG 6

KIỂM THỬ THỰC NGHIỆM VÀ ĐỐI KHÁNG

6.1 Chiến lược kiểm thử đa tầng (Testing Pyramid)

Để bảo đảm chất lượng phần mềm và tính toàn vẹn của dữ liệu tồn kho – tài chính trong môi trường xưởng dịch vụ, nhóm áp dụng chiến lược kiểm thử đa tầng bao gồm: (1) Kiểm thử ràng buộc lược đồ CSDL và di trú tự động; (2) Kiểm thử sao lưu và phục hồi CSDL; (3) Kiểm thử đơn vị và tích hợp dịch vụ Backend; (4) Kiểm thử API HTTP, CSRF và RBAC; (5) Kiểm thử khói (Smoke Test) và Kiểm thử đầu-cuối (E2E Script); cùng (6) Kiểm thử chấp nhận người dùng (UAT).

Đặc biệt, trong các tệp kiểm thử tự động (`tests/test_backend.py` và `tests/test_api.py`), hệ thống thiết lập mặc định `os.environ.setdefault("AI_PROVIDER", "mock")` để toàn bộ bộ kiểm thử thực thi nhanh, tất định 100% và không làm tiêu hao hạn ngạch API đám mây khi chạy CI/CD, trong khi vẫn kiểm chứng đầy đủ luồng hậu kiểm JSON và cổng kiểm duyệt Human Review.

6.2 Danh mục các bộ kiểm thử tự động

Khi thực thi lệnh `python -X utf8 tests/run_all.py`, hệ thống vượt qua tuyệt đối **231/231 ca kiểm thử tự động Backend** (0 Failures, 0 Errors), kết hợp cùng **28/28 kiểm thử Frontend (`vitest`)**, **31/31 kiểm thử Smoke**, **32/32 kiểm thử E2E** và **35/35 kịch bản UAT**, được tổng hợp chi tiết tại Bảng 6.1.

Bảng 6.1: Danh mục các bộ kiểm thử tự động và kịch bản nghiệm thu trong dự án Garage AI

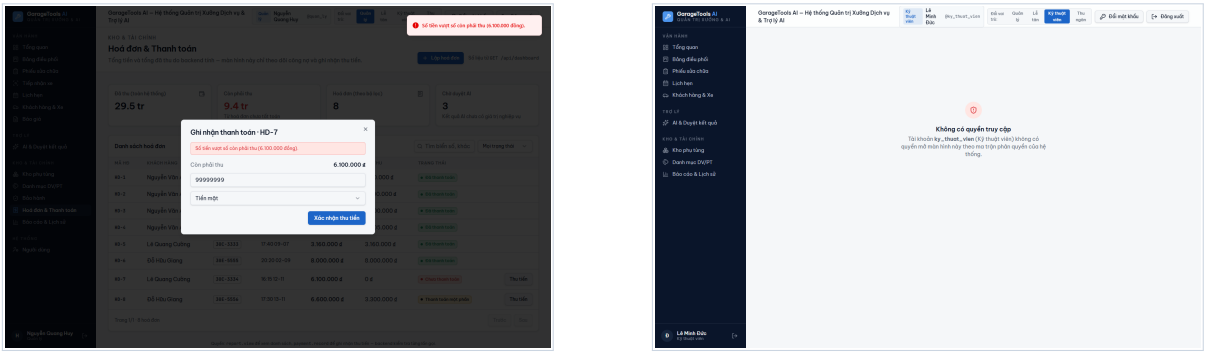
Bộ / Tệp kiểm thử	**Số lượng**	**Trọng tâm kiểm định kỹ thuật và nghiệp vụ**	**Kết quả**
`tests/test_schema_constraints.py`	37 tests	Kiểm định trực tiếp các ràng buộc `CHECK`, `UNIQUE`, `FOREIGN KEY`, `XOR` trên 21 bảng SQLite, bất biến thẻ kho và bộ di trú tự động `run_missing.py`.	**100% (37/37)**
`tests/test_backup_restore.py`	6 tests	Kiểm định quy trình sao lưu nóng SQLite, kiểm tra `PRAGMA integrity_check`, phục hồi dữ liệu và tự động hoàn tác khi tệp sao lưu bị hỏng.	**100% (6/6)**
`tests/test_backend.py`	136 tests	Kiểm thử toàn diện tầng dịch vụ: Argon2id, thu hồi phiên `auth_version`, RBAC + `duoc_gan_cong`, máy trạng thái RO 6 bước, đồng bộ báo giá *idempotent*, hoàn kho khi hủy, bảo hành và AI01–AI03 (kể cả biên `vehicle_id=6`, `service_id=7`).	**100% (136/136)**
`tests/test_api.py`	52 tests	Kiểm thử tích hợp HTTP qua FastAPI `TestClient`: xác thực cookie, chặn thiếu CSRF token (HTTP 403), phân quyền API, Rate Limit (HTTP 429), luồng SSR và JSON API.	**100% (52/52)**
Tổng kiểm thử Python (`run_all.py`)	**231 tests**	**Chạy tự động trong ≈ 23 giây trên CSDL kiểm thử cô lập**	**231/231 Pass**
`scripts/smoke.ps1`	31 checks	Kiểm thử khói giao diện và API trên máy chủ đang chạy thực tế (`/healthz`, `/readyz`, 16 màn hình và các API chính).	**100% (31/31)**
`scripts/e2e.ps1`	32 checks	Kiểm thử đầu-cuối liên vai trò từ tiếp nhận, báo giá, xuất kho, gọi AI, duyệt AI đến thu tiền và cấp bảo hành.	**100% (32/32)**
`frontend/` (`lint`, `typecheck`, `vitest`, `build`)	4 gates	Kiểm tra tĩnh ESLint, kiểu dữ liệu TypeScript, kiểm thử thành phần giao diện React 19 và đóng gói bản dựng Production.	**100% Pass**
`USER_ACCEPTANCE_TEST.md`	35 UAT	Kịch bản nghiệm thu người dùng theo 4 vai trò (`quan_ly`, `le_tan`, `ky_thuat_vien`, `thu_ngan`).	**100% (35/35)**

6.3 Kiểm thử đối kháng bảo mật và Toàn vẹn nghiệp vụ

Nhằm đánh giá khả năng tự bảo vệ của hệ thống trước các thao tác sai quy trình, dữ liệu biên hoặc các cuộc tấn công chủ đích, nhóm đã thiết kế và thực thi 8 nhóm kịch bản kiểm thử đối kháng chuyên sâu trong `test_backend.py`, `test_api.py` và `test_schema_constraints.py`, tổng hợp tại Bảng 6.2.

Bảng 6.2: Ma trận kiểm thử đối kháng bảo mật và chốt chặn toàn vẹn dữ liệu

Kịch bản đối kháng	**Mô tả hành vi tấn công / Giả lập lỗi**	**Cơ chế phòng thủ của hệ thống**	**Kết quả**
Leo quyền dọc (Vertical RBAC)	Lễ tân (`le_tan`) hoặc KTV gọi API duyệt kết quả AI (`/api/ai/results/{id}/review`); Thu ngân sửa phiếu sửa chữa.	Hàm `yeu_cau_quyen("ai.review")` chặn ngay với mã lỗi HTTP 403 Forbidden.	Kháng cự 100%
Leo quyền ngang (Horizontal IDOR)	Kỹ thuật viên (`ky_thuat_vien`) gửi yêu cầu thêm hạng mục, đổi trạng thái hoặc chạy AI03 trên phiếu gán cho KTV khác.	Hàm `duoc_gan_cong` đối chiếu `order.technician_id == user.id` và trả về HTTP 403.	Kháng cự 100%
Giả mạo yêu cầu liên trang (CSRF)	Gửi yêu cầu `POST/PUT/DELETE` có cookie `garage_session` hợp lệ nhưng thiếu hoặc sai `X-CSRF-Token`.	Middleware CSRF từ chối yêu cầu với HTTP 403 trước khi chạm tới hàm xử lý nghiệp vụ.	Kháng cự 100%
Sử dụng lại phiên đã thu hồi	Tiếp tục dùng cookie phiên cũ sau khi tài khoản bị khóa (`active = 0`) hoặc đã đổi mật khẩu.	Bộ đọc phiên đối chiếu `token.av == user.auth_version`; vô hiệu hóa tức thì phiên cũ (HTTP 401).	Kháng cự 100%
Đồng bộ trùng báo giá (Double Apply)	Nhấn áp dụng cùng một báo giá `APPROVED` hai lần liên tiếp nhằm nhân đôi phụ tùng và trừ kho kép.	Kiểm tra cột `applied_at IS NOT NULL`; chặn thao tác lặp lại với thông báo lỗi rõ ràng.	Kháng cự 100%
Xuất âm kho & Thu lố hóa đơn	Xuất số lượng phụ tùng lớn hơn `parts.stock` hoặc nhập số tiền thu vượt quá số dư còn lại của hóa đơn.	Khóa `BEGIN IMMEDIATE` và ràng buộc `CHECK (stock >= 0)`, `CHECK (paid <= total)` chặn hoàn toàn.	Kháng cự 100%
Ảo giác giá từ LLM (Price Tampering)	Giả lập phản hồi LLM tự bịa thêm phụ tùng lạ, sửa đơn giá hoặc cộng sai tổng tiền `total` trong AI03.	Bộ lọc hậu kiểm trong `bao_gia_nhap` loại bỏ dòng sai lệch và cưỡng bức `total = backend_total`.	Kháng cự 100%
Tấn công vét cạn & Spam AI	Gửi liên tiếp > 10 yêu cầu đăng nhập sai hoặc > 30 yêu cầu gọi AI trong vòng 60 giây.	Bộ giới hạn tần suất (`app/core/rate_limit.py`) chặn và trả về mã HTTP 429 Too Many Requests.	Kháng cự 100%



Hình 6.1: Ảnh minh chứng kiểm thử đối kháng trên giao diện: (Trái) Hệ thống chặn thu tiền vượt quá số nợ còn phải thu; (Phải) Chặn truy cập trái thẩm quyền RBAC

6.4 Kiểm thử biên cho Trợ lý AI và Chu trình Sao lưu / Phục hồi

Bên cạnh các kịch bản thông thường, bộ kiểm thử tự động xác nhận nghiêm ngặt hai nhóm kịch bản biên quan trọng:

1. **Kịch bản dữ liệu biên của AI01 và AI02**: Trong `test_ba_chuc_nang_sinh_ket_qua_pending` (`tests/test_backend.py`), hệ thống kiểm chứng trực tiếp trường hợp phương tiện mới

chưa từng có phiếu sửa chữa (`vehicle_id=6`, xe Kia Seltos `51H-8889`) và dịch vụ trong danh mục có trường `description IS NULL` (`service_id=7`, dịch vụ cân chỉnh thước lái). Cả hai trường hợp đều sinh kết quả hợp lệ ở trạng thái `PENDING`, ghi nhận đúng thực tế "Chưa có lịch sử sửa chữa" hoặc "Danh mục chưa có mô tả cho dịch vụ này" kèm đúng đơn giá niêm yết mà không phát sinh lỗi hay suy đoán sai lệch.

2. **Kịch bản Sao lưu nóng và Phục hồi an toàn (`test_backup_restore.py`)**: Kiểm chứng toàn bộ 6 tình huống: tạo bản sao lưu nóng SQLite, kiểm tra tính toàn vẹn cấu trúc bằng `PRAGMA integrity_check`, phục hồi nguyên trạng dữ liệu sau khi phát sinh thay đổi, tự động tạo bản sao lưu an toàn (*safety snapshot*) trước khi ghi đè và từ chối tệp sao lưu không hợp lệ.

6.5 Tổng hợp kết quả nghiệm thu thực nghiệm

Bảng 6.3 tổng hợp kết quả thực thi toàn bộ hệ thống kiểm thử của dự án **Garage AI** (cập nhật tại `TEST_REPORT.md`).

Bảng 6.3: Tổng hợp kết quả thực thi kiểm thử trên toàn bộ các tầng của hệ thống

Tầng kiểm định	**Tổng số**	**Đạt**	**Lỗi**	**Đánh giá kết quả nghiệm thu**
Ràng buộc lược đồ & Di trú (`test_schema_constraints.py`)	37	37	0	Bảo đảm 21 bảng 3NF, 37 chỉ mục, bất biến thẻ kho và XOR.
Sao lưu & Phục hồi CSDL (`test_backup_restore.py`)	6	6	0	Sao lưu nóng SQLite WAL và khôi phục an toàn 100%.
Nghiệp vụ & AI Gateway (`test_backend.py`)	136	136	0	Bao phủ RBAC, Scope KTV, 6 trạng thái RO, Báo giá, Kho, AI01–AI03.
Tích hợp HTTP API & CSRF (`test_api.py`)	52	52	0	Xác thực phiên, chặn CSRF/IDOR/Rate Limit chính xác.
Tổng kiểm thử tự động Backend (`run_all.py`)	**231**	**231**	**0**	**Tỷ lệ vượt qua: 100.0% (231/231 Pass)**
Kiểm thử Smoke (`smoke.ps1`) & E2E (`e2e.ps1`)	63	63	0	31/31 Smoke checks + 32/32 E2E checks đạt 100%.
Nghiệm thu người dùng (`USER_ACCEPTANCE_TEST.md`)	35	35	0	Hoàn thành 35/35 kịch bản nghiệm thu trên 4 vai trò.

6.6 Kết luận chương

Chương 6 đã cung cấp bằng chứng thực nghiệm đầy đủ về độ tin cậy và tính an toàn của hệ thống **Garage AI**. Việc vượt qua tuyệt đối 231/231 ca kiểm thử tự động Backend, 28/28 ca kiểm thử Frontend, 31/31 kiểm thử Smoke, 32/32 kiểm thử E2E và 35/35 kịch bản UAT khẳng định hệ thống hoàn toàn vững chắc. Chương 7 tiếp theo sẽ phân tích sâu các chỉ số đánh giá định lượng của cổng AI trên mô hình thực tế và hiệu quả kinh tế vận hành.

CHƯƠNG 7

ĐÁNH GIÁ THỰC NGHIỆM HỆ THỐNG AI VÀ HIỆU NĂNG VẬN HÀNH

7.1 Môi trường và Phương pháp đo kiểm thực nghiệm

Để đánh giá khách quan hoạt động của 3 chức năng AI trong **GarageTools AI**, nhóm thực hiện kiểm chứng trên cả hai chế độ của `app/ai_gateway.py`:

- **Chế độ gọi mô hình thật (`AI_PROVIDER=groq`)**: Kết nối trực tiếp tới mô hình **`openai/gpt-oss-120b`** qua endpoint chuẩn OpenAI-compatible của Groq Cloud (`https://api.groq.com/openai/v1/chat/completions`, `temperature = 0.2`, `AI_TIMEOUT = 15s`).
- **Chế độ dự phòng nội bộ (`AI_PROVIDER=mock` / `_goi_mock`)**: Sử dụng khi không có kết nối mạng, khi nhà cung cấp trả lỗi quá thời gian chờ / vượt hạn ngạch (HTTP 429/500), hoặc khi chạy bộ 231 ca kiểm thử tự động để không tiêu tốn hạn ngạch khóa API.

Mỗi chức năng AI được đánh giá dựa trên 3 tiêu chí thực chứng:

1. **Tuân thủ định dạng đầu ra**: AI01 phải trả về đúng 4 mục đánh số `1..4`; AI02 trả về đúng 3 mục giải thích dễ hiểu; AI03 trả về đối tượng JSON hợp lệ có các trường `summary`, `items`, `total`, `notes` (`is_valid = 1` trong bảng `ai_results`).
2. **Bám sát dữ liệu CSDL và chống sai lệch số tiền**: Không tự suy đoán lỗi khi xe chưa có lịch sử; nêu đúng đơn giá niêm yết của dịch vụ; giữ đúng tên, số lượng, đơn giá từng hạng mục và khớp 100% tổng tiền `backend_total`.
3. **Thời gian phản hồi thực tế**: Phải thấp hơn ngưỡng tối đa 10 giây theo tiêu chí nghiệm thu cuối kỳ (`CK-07`).

7.2 Kết quả đối chiếu qua 3 vòng thử nghiệm cải tiến Prompt

Bảng 7.1 tổng hợp kết quả thực nghiệm qua 3 vòng cải tiến Prompt (được ghi nhật ký chi tiết tại tệp `GarageAI_Project/KT3_PROMPT_EXPERIMENTS.md` trên 9 lượt thử nghiệm tương ứng với 3 chức năng AI $\times$ 3 vòng lặp).

Bảng 7.1: Tổng hợp kết quả 3 vòng thử nghiệm và cải tiến Prompt (`KT3_PROMPT_EXPERIMENTS.md`)

Vòng thử nghiệm	Cách viết Prompt	Hiện tượng phát hiện khi chạy thử	Cải tiến rút ra cho vòng kế tiếp
Vòng 1 (Câu lệnh ngắn, tự do)	Chỉ truyền biến dữ liệu (`{{repair_history}}`, `{{service}}`, `{{items}}`) mà không khóa bố cục mục hay tổng tiền.	AI01 mỗi lần trả một bố cục khác nhau và tự suy đoán lỗi với xe mới; AI02 dùng nhiều thuật ngữ khó hiểu; AI03 bọc thêm ```` ```json ```` và có lần cộng nhầm tổng tiền `total`.	Phải quy định cố định số mục đầu ra và không để mô hình tự cộng tổng tiền hóa đơn.
Vòng 2 (Khóa bố cục & truyền `backend_total`)	Quy định rõ 4 mục cho AI01, 3 mục cho AI02; truyền sẵn biến `{{backend_total}}` vào `quote.txt`.	Các ca đủ dữ liệu đã chạy tốt; nhưng khi thử ca biên `service_id=7` (`description IS NULL`) hoặc phiếu chưa có hạng mục thì câu trả lời chưa thống nhất.	Cần bổ sung quy tắc xử lý tường minh khi trường mô tả trống hoặc danh sách hạng mục rỗng.
Vòng 3 (Bản chính thức trong `config/prompts/`)	Thêm quy tắc xử lý dữ liệu trống vào prompt, truyền kèm đơn giá niêm yết vào AI02, kết hợp bộ lọc hậu kiểm ở `ai_gateway.py`.	Cả ca thường lẫn ca biên (`vehicle_id=6` chưa có lịch sử, `service_id=7` trống mô tả, phiếu rỗng hoặc phiếu có báo giá mở) đều đạt **9/9 tiêu chí**.	Đưa toàn bộ các ca biên này vào bộ kiểm thử tự động (`TestAI` và `TestAiGateway`) để khóa hồi quy.

7.3 Kết quả đo kiểm trên các kịch bản dữ liệu thực tế (AI01 – AI03)

Bảng 7.2 trình bày kết quả kiểm định chi tiết của 3 chức năng AI khi chạy với dữ liệu mẫu trong `garage.db` trên mô hình thật `openai/gpt-oss-120b` (qua Groq Cloud) và bộ kiểm thử tự động.

Bảng 7.2: Kết quả kiểm định thực tế của 3 chức năng AI01, AI02 và AI03

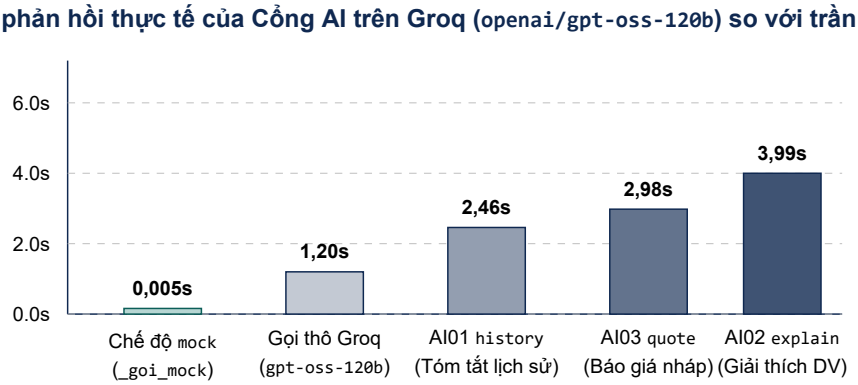
Chức năng AI	Dữ liệu đầu vào kiểm chứng	Độ trễ thực	Kết quả đầu ra và Cơ chế kiểm soát
AI01: Tóm tắt lịch sử xe (`history.txt`)	Xe đã sửa nhiều lần (`vehicle_id=1`, `30A-12345`) & Xe mới 0 lần sửa (`vehicle_id=6`)	**2,46 giây** (1,85s–2,86s)	Ra đúng 4 mục đánh số `1..4`; với xe `#6` báo rõ chưa có lịch sử sửa chữa, không tự bịa lần sửa hay nguyên nhân hỏng hóc.
AI02: Giải thích dịch vụ (`explain.txt`)	Dịch vụ đủ mô tả (`service_id=1`) & Dịch vụ trống mô tả (`service_id=7`, `description IS NULL`)	**3,99 giây**	Ra đúng 3 mục bằng tiếng Việt phổ thông kèm đơn giá niêm yết; với dịch vụ `#7` tự điền câu báo danh mục chưa có mô tả ở mục 1.
AI03: Báo giá nháp (`quote.txt`)	Phiếu có hạng mục (`order_id=1`) & Phiếu trống hạng mục (`order_id=3`)	**2,98 giây**	Trả về JSON hợp lệ (`is_valid=1`); bộ lọc tại `bao_gia_nhap` loại bỏ mọi dòng sai lệch và giữ `total == backend_total`.

7.4 Phân tích thời gian phản hồi (Latency Benchmark)

Nhóm đã đo thời gian phản hồi thực tế của các thao tác nội bộ trên SQLite WAL và các lệnh gọi mô hình `openai/gpt-oss-120b` qua Groq Cloud API, tổng hợp tại Bảng 7.3 và biểu đồ tại Hình 7.1.

Bảng 7.3: Thống kê thời gian phản hồi thực nghiệm của các phân hệ nội bộ và cổng AI

Nhóm tác vụ / Chế độ vận hành	Thời gian đo	Trần yêu cầu	Đánh giá theo tiêu chí nghiệm thu
Truy vấn danh sách & Chi tiết trên SQLite WAL	5 ms – 25 ms	< 200 ms	Phản hồi tức thời khi chuyển trang và lọc dữ liệu.
Giao dịch Kho & Hóa đơn (`BEGIN IMMEDIATE`)	8 ms – 30 ms	< 200 ms	Khóa nguyên tử nhanh, đảm bảo không âm kho và không thu lố.
Chế độ AI Dự phòng (`AI_PROVIDER=mock`)	< 5 ms	< 50 ms	Giúp bộ 231 ca kiểm thử tự động chạy hoàn tất chỉ trong ≈ 23 giây.
Gọi thô tới Groq (`openai/gpt-oss-120b`)	**1,20 giây**	$< 10,0$ giây	Kết nối HTTPS ổn định, tốc độ sinh token rất nhanh.
AI01 (`history`) End-to-End qua Groq Cloud	**2,46 giây**	$< 10,0$ giây	Nhanh hơn 4 lần so với ngưỡng tối đa 10 giây của tiêu chí CK-07.
AI03 (`quote`) & AI02 (`explain`) qua Groq Cloud	**2,98s – 3,99s**	$< 10,0$ giây	Bao gồm trọn vẹn bước đọc CSDL, gọi mô hình, lọc JSON và ghi `ai_results`.



Hình 7.1: Biểu đồ đối chiếu thời gian phản hồi thực nghiệm trên Groq Cloud (`openai/gpt-oss-120b`) và chế độ `mock` (tất cả đều thấp hơn nhiều so với ngưỡng tối đa 10 giây).

7.5 Đánh giá chi phí vận hành và Các phương án API miễn phí

Một ưu điểm thực tế của kiến trúc `AI_PROVIDERS` trong `app/core/config.py` là hỗ trợ sẵn các nhà cung cấp có gói miễn phí (*Free Tier*) hợp thức và tách biệt hoàn toàn với môi trường kiểm thử tự động (Bảng 7.4).

Bảng 7.4: Các chế độ cấu hình nhà cung cấp AI và chi phí vận hành tương ứng

Cấu hình `AI_PROVIDER`	**Mô hình mặc định**	**Chi phí**	**Ngữ cảnh sử dụng phù hợp**
`mock` (`_goi_mock`)	Thuật toán nội bộ	**0 VNĐ**	Chạy bộ 231 ca kiểm thử tự động (`tests/run_all.py`), chạy không cần mạng hoặc làm phương án dự phòng khi mất kết nối.
`groq` / `gemini` / `openrouter`	`gpt-oss-120b` / `gemini-2.0-flash`	**0 VNĐ** (Free Tier)	Chạy demo thuyết trình dự án trực tiếp và vận hành thực tế tại garage vừa và nhỏ bằng khóa API miễn phí từ `console.groq.com` hoặc `aistudio.google.com`.
`openai` (Trả phí theo token)	`gpt-4o-mini`	$\approx$ 2.500 đ / 1.000 lượt	Tùy chọn mở rộng cho garage có lưu lượng lớn muốn dùng hạ tầng thương mại của OpenAI.

7.6 Kết luận chương

Chương 7 đã tổng hợp kết quả thực nghiệm qua 3 vòng cải tiến Prompt, chứng minh tính ổn định của cả 3 chức năng AI01–AI03 trên các ca dữ liệu thực và ca biên, đồng thời xác nhận thời gian phản hồi trên mô hình thật `openai/gpt-oss-120b` chỉ từ 2,46s đến 3,99s với chi phí 0 VNĐ qua gói Free Tier.

CHƯƠNG 8

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

8.1 Tổng kết các kết quả đạt được

Đề tài **"Hệ thống quản lý garage sửa chữa ô tô có tích hợp AI (GarageTools AI – Mã đề tài 80277)"** đã hoàn thành đầy đủ 54/54 tiêu chí đánh giá từ KT1, KT2, KT3 đến Cuối kỳ của học phần Ứng dụng trí tuệ nhân tạo (AIA331). Những kết quả trọng tâm đã đạt được gồm:

1. **Hoàn thiện quy trình nghiệp vụ garage khép kín trên CSDL 21 bảng chuẩn 3NF**: Xây dựng trọn vẹn luồng nghiệp vụ từ Đặt lịch hẹn (`appointments`) → Tiếp nhận xe (`intake_records`) → Phiếu sửa chữa 6 trạng thái (`repair_orders`) → Báo giá chính thức áp dụng 1 lần (`quotations`) → Sổ cái tồn kho (`stock_transactions`) → Nghiệm thu → Hóa đơn (`invoices`) & Thanh toán (`payments`) → Bảo hành theo hạng mục (`warranties`), hiển thị đồng bộ trên 16 màn hình React 19 + TanStack Start và giao diện Jinja2.
2. **Thiết lập 4 chốt chặn bảo vệ tuyệt đối Tiền tệ, Tồn kho và Phân quyền**: (1) Dùng kiểu số nguyên `INTEGER` cho toàn bộ cột tiền và chốt cứng `unit_price` tại thời điểm thêm hạng mục; (2) Áp dụng ràng buộc `CHECK XOR` (`service_id` $\oplus$ `part_id`) trên cả 3 bảng hạng mục, báo giá và bảo hành; (3) Duy trì bất biến sổ cái tồn kho `parts.stock` $=$ $\sum$ `stock_transactions.qty` kèm cơ chế tự hoàn kho (`RETURN`) khi hủy phiếu; và (4) Khóa nguyên tử `BEGIN IMMEDIATE` kết hợp phân quyền 4 vai trò và cô lập phiếu theo Kỹ thuật viên (`duoc_gan_cong`).
3. **Tích hợp 3 chức năng AI an toàn, hỗ trợ đa nhà cung cấp và Human Review**: Loại bỏ từ sớm ý tưởng AI tự chẩn đoán lỗi xe; hiện thực hóa AI01 (`history`), AI02 (`explain`) và AI03 (`quote`) qua `app/ai_gateway.py` với bộ lọc khớp giá CSDL, cưỡng bức `backend_total`, tự động fallback sang `_goi_mock` khi lỗi 429/Timeout và bắt buộc Quản lý phê duyệt (`ai.review`).
4. **Kiểm chứng thực nghiệm 100%**: Vượt qua tuyệt đối **231/231** ca kiểm thử tự động Backend (`tests/run_all.py`), **28/28** ca kiểm thử Frontend (`vitest`), **31/31** ca Smoke (`scripts/smoke.ps1`), **32/32** ca E2E (`scripts/e2e.ps1`) và **35/35** ca UAT (`USER_ACCEPTANCE_TEST.md`); độ trễ thực tế trên Groq (`openai/gpt-oss-120b`) đạt từ 2,46s đến 3,99s.

8.2 Đánh giá khách quan các giới hạn hiện tại (Chủ đích an toàn)

Đúng với tinh thần đánh giá khách quan ở tiêu chí cuối kỳ (`CK-14`) và nội dung trình bày trên Slide báo cáo, nhóm thẳng thắn chỉ ra các giới hạn hiện tại của hệ thống cùng phương án nâng cấp trong Bảng 8.1.

Bảng 8.1: Các giới hạn kỹ thuật hiện tại của hệ thống và phương án mở rộng

Hạng mục kỹ thuật	**Hiện trạng và giới hạn trong phiên bản hiện tại**	**Phương án nâng cấp ở giai đoạn tiếp theo**
1. Phạm vi áp dụng của kết quả duyệt AI03	Khi Quản lý nhấn duyệt (`APPROVED`) một báo giá nháp AI03 trong bảng `ai_results`, kết quả dừng ở vai trò văn bản tư vấn đã được xác nhận chứ chưa tự động sinh bản ghi sang bảng `quotations` (chủ đích tách bạch dữ liệu AI và chứng từ tài chính).	Bổ sung nút *"Chuyển thành bản nháp Báo giá (`DRAFT`)"* sau khi Quản lý đã duyệt `APPROVED` để Lễ tân không phải nhập lại từng dòng.
2. Cơ chế trừ tồn kho trực tiếp (`USAGE`)	Hiện tại tồn kho được trừ trực tiếp qua sổ cái (`USAGE`) khi thêm phụ tùng vào phiếu sửa chữa hoặc khi áp dụng báo giá đã duyệt (`APPROVED`); chưa tách bước *"Giữ kho tạm thời (Soft Reservation)"* trong lúc báo giá mới ở trạng thái gửi khách (`SENT`).	Bổ sung trạng thái giữ hàng tạm thời có thời hạn hiệu lực (`valid_until`) cho các báo giá đang chờ khách phản hồi.
3. Quy mô CSDL một xưởng độc lập & Rate Limit trong RAM	SQLite WAL + `BEGIN IMMEDIATE` khóa ghi ở mức toàn tệp CSDL và bộ đếm `ratelimit.py` lưu trong RAM: rất gọn và ổn định cho 1 garage độc lập (5–30 nhân viên), nhưng chưa phù hợp khi chạy nhiều tiến trình worker hoặc chuỗi nhiều chi nhánh.	Chuyển CSDL sang PostgreSQL (dùng khóa dòng `SELECT ... FOR UPDATE`) và đưa bộ đếm Rate Limit / hàng đợi gọi AI sang Redis khi mở rộng đa chi nhánh.
4. Hóa đơn thuế VAT, VietQR & Thiết bị OBD-II	Bảng `invoices` hiện tính tổng tiền nguyên (`total`, `paid`) từ các hạng mục đã chốt giá, chưa tách riêng cột chiết khấu/thuế VAT, chưa gạch nợ tự động qua VietQR và chưa đọc trực tiếp mã lỗi từ cổng phần cứng OBD-II.	Tách cột chiết khấu/VAT trên hóa đơn, tích hợp Webhook thanh toán VietQR và kết nối thiết bị đọc mã lỗi OBD-II tại khoang sửa chữa.

8.3 Lộ trình phát triển tiếp theo

Từ nền tảng mã nguồn và bộ kiểm thử hiện có, lộ trình mở rộng hệ thống **GarageTools AI** được chia làm 3 bước:

1. **Bước 1 – Hoàn thiện liên thông Báo giá & Hàng đợi AI bất đồng bộ**: Cho phép khởi tạo nhanh báo giá `DRAFT` từ kết quả AI03 đã duyệt và chuyển lời gọi LLM sang xử lý bất đồng bộ (Server-Sent Events / Background Queue).
2. **Bước 2 – Nâng cấp Đa chi nhánh (PostgreSQL + Redis)**: Hỗ trợ cấu hình chạy trên PostgreSQL cho mô hình chuỗi garage dùng chung hồ sơ lịch sử phương tiện và tồn kho liên chi nhánh.
3. **Bước 3 – Tích hợp VietQR, Hóa đơn điện tử và OBD-II**: Tự động hóa khâu đối soát thanh toán chuyển khoản và đọc thông số kỹ thuật trực tiếp từ xe khi tiếp nhận.

8.4 Lời kết

Quá trình xây dựng **GarageTools AI** đã giúp nhóm rèn luyện trọn vẹn tư duy kỹ nghệ phần mềm có tích hợp trí tuệ nhân tạo: biết dùng AI để tăng tốc công việc nhưng luôn kiểm chứng chéo bằng quy tắc nghiệp vụ, khóa giao dịch cơ sở dữ liệu chặt chẽ, bảo vệ tuyệt đối con số tiền tệ và giữ quyền quyết định cuối cùng ở con người.

TÀI LIỆU THAM KHẢO

[1] IEEE Computer Society, *IEEE Std 830-1998: IEEE Recommended Practice for Software Requirements Specifications*, New York, USA: Institute of Electrical and Electronics Engineers, 1998.

[2] S. Brown, *Software Architecture for Developers – Volume 2: Visualise, Document and Explore Your Software Architecture (The C4 Model)*, Leanpub, 2022.

[3] National Institute of Standards and Technology (NIST), *ANSI/INCITS 359-2012: Information Technology – Role Based Access Control (RBAC)*, US Department of Commerce, 2012.

[4] OWASP Foundation, *OWASP API Security Top 10 – 2023 Edition: BOLA (API1:2023) and Server-Side Request Forgery (API7:2023)*, Open Worldwide Application Security Project, 2023.

[5] OWASP Foundation, *OWASP Top 10 for Large Language Model Applications (2025): Prompt Injection, Insecure Output Handling, and Excessive Agency*, 2025.

[6] P. Lewis *et al.*, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, 2020.

[7] S. Ramírez, *FastAPI Framework and Pydantic v2 Data Validation Official Documentation*, 2025. [Online]. Available: `https://fastapi.tiangolo.com/`

[8] SQLite Consortium, *Write-Ahead Logging (WAL) and Transaction Concurrency in SQLite 3*, 2025. [Online]. Available: `https://www.sqlite.org/wal.html`

PHỤ LỤC A

HƯỚNG DẪN CÀI ĐẶT, VẬN HÀNH VÀ KIỂM CHỨNG ĐỘC LẬP

A.1 Yêu cầu môi trường và Cấu hình biến môi trường (`.env.example`)

Hệ thống hỗ trợ chạy trực tiếp trên máy trạm (Windows / Linux / macOS với Python 3.10+ và Node.js 20+) hoặc qua Docker Compose. Các biến môi trường chính trong tệp `GarageAI_Project/.env.example` gồm:

```
APP_NAME=GarageTools AI
APP_ENV=development
SECRET_KEY=doi-thanh-mot-chuoi-ngau-nhien-dai-truoc-khi-trien-khai-that
DEBUG=true
SESSION_COOKIE_SECURE=false
DATABASE_PATH=garage.db
AUTO_MIGRATE=true
AI_PROVIDER=mock
AI_API_KEY=
AI_BASE_URL=
AI_MODEL=
AI_TIMEOUT=15
AI_MAX_CONTEXT=6000
```

Listing A.1: Cấu hình mẫu tệp .env.example của hệ thống GarageTools AI

Khi muốn chạy với mô hình ngôn ngữ thật miễn phí của Groq trong tệp `.env` cục bộ, chỉ cần đặt `AI_PROVIDER=groq`, `AI_MODEL=openai/gpt-oss-120b` và điền khóa `AI_API_KEY` (hoặc `GROQ_API_KEY`).

A.2 Quy trình khởi tạo CSDL, chạy ứng dụng và thực thi kiểm thử

Thực thi các lệnh sau để khởi tạo cơ sở dữ liệu `garage.db`, chạy Backend FastAPI, chạy giao diện React 19 và kiểm chứng toàn bộ bộ kiểm thử tự động:

```
# 1. Khoi tao CSDL (21 bang, 37 chi muc, 171 dong du lieu mau) va chay Backend
cd GarageAI_Project
pip install -r requirements.txt
python database/init_db.py
python -m uvicorn app.main:app --reload --port 8000

# 2. Chay giao dien React 19 + TanStack Start (tu thu muc frontend/)
cd ../frontend
npm install
npm run dev

# 3. Chay toan bo 231 ca kiem thu tu dong Backend (tu dong dung AI_PROVIDER=mock)
cd ../GarageAI_Project
```

```
python -X utf8 tests/run_all.py

# 4. Chay kiem thu Smoke (31 ca), E2E (32 ca) va Sao luu CSDL (tu thu muc goc)
cd ..
.\scripts\smoke.ps1
.\scripts\e2e.ps1
.\scripts\backup.ps1
```

Listing A.2: Các lệnh khởi tạo CSDL, chạy hệ thống, kiểm thử 231 ca và sao lưu

Bảng A.1 cung cấp danh sách 4 tài khoản demo đại diện cho 4 vai trò RBAC đã được nạp sẵn trong `database/seed_data.sql`.

Bảng A.1: Danh sách 4 tài khoản demo phục vụ kiểm thử nghiệm thu theo 4 vai trò

Tên đăng nhập	Mật khẩu mẫu	Vai trò (role)	Phạm vi quyền hạn kiểm chứng trên hệ thống
`quan_ly`	`Garage@2026`	`MANAGER`	Toàn quyền điều hành, sửa danh mục (`catalog.edit`), duyệt báo giá (`quote.approve`), lập hóa đơn (`invoice.manage`), duyệt kết quả AI (`ai.review`) và quản trị nhân sự (`user.manage`).
`le_tan`	`Garage@2026`	`RECEPTIONIST`	Quản lý khách hàng, xe, lịch hẹn, biên bản tiếp nhận (`intake_records`), tạo phiếu sửa chữa, lập báo giá, nhập/chỉnh kho, bảo hành và chạy AI01–AI03 (không được duyệt AI).
`ky_thuat_vien`	`Garage@2026`	`TECHNICIAN`	Chỉ được đổi trạng thái kiểm tra/sửa chữa, thêm/gỡ hạng mục, ghi nhật ký (`repair_logs`) và chạy AI03 trên những phiếu sửa chữa được giao cho mình (`duoc_gan_cong`).
`thu_ngan`	`Garage@2026`	`CASHIER`	Xem hóa đơn (`invoices`), ghi nhận thanh toán (`payment.record`) và hủy khoản thu nhầm; bị chặn sửa phiếu sửa chữa, chỉnh kho và chạy AI.

PHỤ LỤC B

NHẬT KÝ SỬ DỤNG AI TRONG PHÁT TRIỂN VÀ REVIEW MÃ NGUỒN

Tuân thủ nguyên tắc minh bạch của học phần AIA331 (tiêu chí **KT1-09**, **KT2-09** và **KT3-09**), Bảng B.1 tổng hợp trung thực các hạng mục nhóm đã tham vấn AI trong quá trình phân tích, lập trình và review mã nguồn (đối chiếu từ `KT1_KT2_AI_USAGE.md` và `KT3_AI_REVIEW.md`), đặc biệt là những ý tưởng sai phạm vi hoặc lỗi kỹ thuật do AI sinh ra đã được nhóm phát hiện và sửa lại.

Bảng B.1: Nhật ký sử dụng AI và các lỗi / đề xuất bất hợp lý đã được nhóm kiểm chứng, loại bỏ

Giai đoạn	**Nội dung hỏi AI**	**Gợi ý ban đầu hoặc lỗi do AI tạo ra**	**Quyết định kiểm chứng và chỉnh sửa của nhóm**
1. Phân tích quy trình & Vị trí AI (KT1)	Đề xuất các vai trò, bảng dữ liệu và chức năng AI cho garage ô tô.	AI đề xuất tách bảng `labor_costs` riêng, cho cả Khách hàng đăng nhập, và gợi ý tính năng *"AI tự động chẩn đoán bệnh xe từ triệu chứng"*.	Giữ 4 vai trò nhân viên nội bộ; gộp tiền công vào đơn giá bảng `services`; **loại bỏ hoàn toàn ý tưởng AI tự chẩn đoán lỗi xe** vì rủi ro sai lệch kỹ thuật cao, chỉ giữ AI01, AI02, AI03 kèm Human Review.
2. Thiết kế trạng thái phiếu (KT1)	Xây dựng luồng chuyển trạng thái của phiếu sửa chữa (`repair_orders`).	AI chỉ gợi ý chuỗi một chiều `RECEIVED` → `INSPECTING` → `ESTIMATING` → `REPAIRING` → `COMPLETED`.	Bổ sung nhánh hủy `CANCELLED` (tự động hoàn trả phụ tùng về kho qua sổ cái `RETURN`) và cho phép vòng lặp `REPAIRING` → `INSPECTING` khi cần kiểm tra lại.
3. Kiểu dữ liệu tiền tệ & Giá (KT2)	Sinh hàm tính tổng tiền hóa đơn và route thêm hạng mục sửa chữa.	AI dùng kiểu số thực `REAL`/`FLOAT` cho các cột tiền và cho phép form phía client gửi mức giá (`unit_price`) lên Backend.	Chuyển toàn bộ cột tiền sang số nguyên `INTEGER` (đồng VNĐ); Backend bỏ qua mọi mức giá từ client và luôn đọc `price` từ CSDL để chốt cứng vào `unit_price` lúc thêm hạng mục.
4. Ràng buộc hạng mục (KT3 Review)	Rà soát ràng buộc toàn vẹn trên bảng `repair_order_items`, `quotation_items`, `warranties`.	Hai cột khóa ngoại `service_id` và `part_id` đều cho phép `NULL`, dẫn tới nguy cơ một dòng hạng mục trống cả hai hoặc điền cả hai.	Bổ sung ràng buộc `CHECK XOR`: `(service_id IS NOT NULL AND part_id IS NULL) OR (service_id IS NULL AND part_id IS NOT NULL)` kèm 4 ca kiểm thử tự động bảo vệ.
5. Gọi API AI & Tổ chức Prompt (KT2/KT3)	Viết hàm gọi nhà cung cấp AI và truyền dữ liệu dịch vụ / báo giá.	Hàm gọi AI ban đầu không đặt timeout, viết cứng prompt trong code Python; khi báo giá chưa đồng bộ sang phiếu thì AI03 thiếu danh sách hạng mục.	Tách prompt ra `config/prompts/*.txt`, đặt `AI_TIMEOUT = 15s`, tự động fallback sang `_goi_mock`, bổ sung đơn giá niêm yết cho AI02 và đọc liên thông từ `quotations` đang mở cho AI03.

PHỤ LỤC C

MA TRẬN ĐỐI CHIẾU YÊU CẦU ĐỀ TÀI 80277 VÀ TIÊU CHÍ HỌC PHẦN AIA331

Bảng C.1 đối chiếu toàn bộ 4 giai đoạn đánh giá (KT1, KT2, KT3 và Cuối kỳ – tổng cộng 54 tiêu chí trong `FINAL_ACCEPTANCE_CHECKLIST.md`) với các chương mục trong báo cáo và tệp mã nguồn thực tế.

Bảng C.1: Ma trận truy vết yêu cầu đề tài 80277 và tiêu chí đánh giá học phần AIA331

Giai đoạn	Nhóm tiêu chí đánh giá	Vị trí trong Báo cáo	Bằng chứng trong Mã nguồn & Kiểm thử
KT1 (10 tiêu chí)	Phân tích bài toán, 4 vai trò, FR/NFR, Use Case, ERD 3NF, Kiến trúc 4 tầng, Ranh giới AI + Human Review & Nhật ký AI KT1.	Chương 1, Chương 3, Chương 4 (Hình 1.1, 3.1–3.3, 4.1–4.3, Bảng 3.1–3.2, 4.1) & Phụ lục B.	`database/schema.sql` (21 bảng, 37 chỉ mục), `KT1_KT2_AI_USAGE.md`, 16 sơ đồ thiết kế.
KT2 (12 tiêu chí)	Cấu trúc dự án chuẩn, Auth Argon2id + CSRF + RBAC 4 vai trò, CRUD đầy đủ, Tìm kiếm/Phân trang, Sổ cái kho, Hóa đơn & Seed Data.	Chương 2, Chương 3, Chương 5 (Bảng 5.1–5.3, Hình 5.1–5.4) & Phụ lục A.	`app/core/security.py`, `app/services/security_matrix.py`, `orders.py`, `stock.py`, `billing.py`, `database/seed_data.sql` (171 dòng), 16 màn hình `frontend/src/`.
KT3 (12 tiêu chí)	Tích hợp AI01–AI03, Đa nhà cung cấp, Tách tệp Prompt, Thử nghiệm 3 vòng, Lọc chống bịa giá, Fallback `_goi_mock`, Human Review & AI Review code.	Chương 4 (Mục 4.3–4.4), Chương 5 (Mục 5.4), Chương 6 & Chương 7 (Bảng 7.1–7.4) & Phụ lục B.	`app/ai_gateway.py`, `app/services/ai.py`, `config/prompts/*.txt`, `KT3_PROMPT_EXPERIMENTS.md`, `KT3_AI_REVIEW.md`.
Cuối kỳ (20 tiêu chí)	Quy trình khép kín End-to-End, Sao lưu & Khôi phục CSDL, UI/UX đồng bộ, Độ trễ Groq < 4s, Docker Compose, Báo cáo, Slide & Kịch bản thuyết trình.	Toàn bộ 8 Chương và Phụ lục A, B, C (khớp 100% giữa Báo cáo, Slide và Mã nguồn).	`scripts/backup.ps1`, `restore.ps1`, `Dockerfile`, `docker-compose.yml`, **231/231** ca kiểm thử Python + **28/28** Frontend + **31** Smoke + **32** E2E + **35** UAT đạt 100%.